

FIT3162 Computer Science Project 2

Semester 1, 2025

Final Project Report

Enhancing Rebar Supply Chain Efficiency with AI-Driven Counting

Project Team MCS03

Chew Chee Lel	33560226	cche0262@student.monash.edu
Chin Wei Han	33368406	wchi0049@student.monash.edu
Ong Chang Le	33253935	cong0026@student.monash.edu
Tan Wei Ming	33617635	wtan0130@student.monash.edu

Project Supervisor

Dr. Tan Chee Keong

Word Count

10969

Table of Contents

1. Introduction.....	4
1.1 Project Motivation	4
1.2 Project Aims and Objectives	4
1.3 Report Overview and Structure.....	4
2. Project Background.....	5
2.1 Industry Context and Problem Statement	5
2.2 Literature Review.....	5
2.2.1 Traditional Methods and Industry Practices.....	5
2.2.2 Current Automated and AI-Based Approaches	5
2.2.3 Classical Computer Vision for Object Detection	6
2.2.4 Deep Learning for Object Detection.....	7
2.3 Project Rationale and Significance	10
3. Project Outcomes	10
3.1 Overview of Implemented System.....	10
3.1.1 Features and Capabilities	10
3.1.2 Delivered Components.....	11
3.2 Results, Achievements, and Requirement Satisfaction	11
3.2.1 Functional Results and Requirement Coverage	11
3.2.2 Non-Functional Results and Requirement Coverage	12
3.2.3 Requirements Traceability Matrix	13
3.3 Key Decisions and Justifications	13
3.4 Comparison to Existing Solutions / State of the Art	14
3.5 Limitations and Challenges.....	15
3.6 Potential Improvements and Future Work	15
4. Methodology.....	16
4.1 System Design and Architecture	16
4.1.1 Final System Architecture	16
4.1.2 Component Breakdown.....	17
4.1.3 Data Flow and Integration.....	18
4.2 Software Tools and Technologies	19
4.3 Model Development and Implementation.....	20
4.3.1 Data Collection	20
4.3.2 Semi-Automated Dataset Annotation.....	20
4.3.3 Initial Model Training and Selection.....	21

4.3.4 Model Architecture	22
4.4 Implementation Activities and Process	24
4.4.1 Development Workflow	24
4.4.2 Testing and Validation Approach	25
4.5 Deviation from Initial Design	26
5. Software Deliverables	27
5.1 Deliverables Overview	27
5.2 Visual Overview of System and Features	28
5.2.1 Mobile Application	28
5.2.2 Model	30
5.2.3 Backend and Cloud Infrastructure	30
5.3 Software Quality Assessment	32
5.3.1 Robustness and Reliability	32
5.3.2 Security and Data Privacy	33
5.3.3 Usability	33
5.3.4 Scalability	33
5.3.5 Portability	34
5.3.6 Maintainability	34
5.4 Sample Source Code	35
6. Software and Project Critique	35
6.1 Evaluation of Project Execution	35
6.2 Outcome vs. Proposal and Deviations	36
6.2.1 Alignment with Original Proposal	36
6.2.2 Key Deviations and Their Rationale	36
6.3 Project Management Reflection	37
6.4 Lessons Learned and Recommendations	38
7. Conclusion	39
7.1 Summary of Key Findings	39
7.2 Overall Evaluation of Project Outcomes	39
7.3 Final Thoughts and Recommendations	39
8. Appendices	40
9. References	48

1. Introduction

1.1 Project Motivation

Accurate inventory management is a cornerstone of the steel construction industry, where reinforcing bars (rebars) are essential components for structural integrity in buildings and infrastructure. Despite advances in construction technology, the process of counting rebars remains largely manual and labour-intensive. This reliance on human counting introduces risks of error, inefficiency, and delays throughout critical stages such as manufacturing, warehousing, and on-site delivery. Manual counting of rebar is inherently error-prone due to fatigue, distraction, and repetitive task nature, as extensively documented in human factors research (Reason, 1990). Errors in rebar inventory can result in costly project overruns, compromised structural safety, and diminished supply chain transparency. With increasing industry demand for transparency, efficiency, and error reduction, there is a growing need for an automated, reliable solution that can deliver rapid and precise rebar counts under diverse, real-world conditions.

1.2 Project Aims and Objectives

The primary aim of this project is to transform the traditional process of rebar counting by developing an AI-driven system that leverages deep learning and computer vision techniques. The core objective is to deliver a mobile application, integrated with a robust backend, capable of automatically detecting and counting rebars in images with an accuracy of at least 99%. This system is expected to return results in under 5 seconds per image, supporting real-time decision-making in industrial environments. The solution must demonstrate reliability across a wide range of practical scenarios, including varying lighting and backgrounds. To achieve this, the project focuses on several key objectives:

- **Developing and fine-tuning a deep learning model** for accurate rebar detection and counting, trained on diverse datasets from both industry and public sources.
- **Designing a user-friendly Android mobile application** that enables efficient image capture, upload, annotation, and result visualization for end-users.
- **Building a secure, scalable backend infrastructure** to handle image processing, data storage, and user authentication with strong performance and reliability.
- **Implementing comprehensive testing, evaluation, and usability assessment** to ensure the system meets its accuracy, speed, and robustness targets.

By achieving these objectives, the project seeks to set a new standard for efficiency and reliability in rebar supply chain management.

1.3 Report Overview and Structure

This final project report presents a comprehensive account of the design, development, and evaluation of the AI-driven rebar counting system. It is organized as follows:

- **2. Project Background** – Introduces the industry context, outlines current challenges, and presents an updated literature review on automated counting and deep learning approaches, building upon the initial project proposal.
- **3. Project Outcomes** – Details the main results and achievements, explains how user requirements were addressed, and discusses the successes, limitations, and improvement areas.
- **4. Methodology** – Describes the design and implementation process, software and tools used, and any deviations from the initial proposal, providing sufficient detail for reproducibility.

- **5. Software Deliverables** – Summarizes all deliverables, presents illustrative screenshots, and discusses the software’s quality characteristics.
- **6. Software and Project Critique** – Reflects on project management, execution, team collaboration, and lessons learned, comparing actual outcomes to initial expectations.
- **7. Conclusion** – Recaps the key contributions and outcomes and provides an overall evaluation of the project’s success.
- **8. Appendices** – Includes supplementary materials such as the RTM, sample source code, and other relevant documentation.
- **9. References** – Lists all sources cited, following APA guidelines.

Through this structure, the report aims to provide a clear, logical, and evidence-based account of the entire project journey, from identifying the industry problem to delivering a validated, deployable solution. It is intended for both technical and non-technical readers, including stakeholders, academic assessors, and potential industry adopters.

2. Project Background

2.1 Industry Context and Problem Statement

The construction industry, specifically the steel sector, extensively utilizes reinforcing bars (rebars) for ensuring structural integrity in construction projects. Despite technological advances, rebar counting processes remain predominantly manual, requiring multiple human verifications across manufacturing, warehousing, and delivery phases. These manual processes introduce errors, delays, and inefficiencies, negatively impacting supply chain transparency and accuracy.

2.2 Literature Review

2.2.1 Traditional Methods and Industry Practices

Current industry practices for rebar counting often rely on human workers physically counting rebars and marking them with stickers or tags to avoid double counting. This process is labor-intensive, time-consuming, and susceptible to human error. This process is labor-intensive, time-consuming, and susceptible to human error. Workers may make errors when consolidating their counts or when coordinating the findings of different counters, often requiring a complete recount when errors are detected, which further increases time and labor requirements.

2.2.2 Current Automated and AI-Based Approaches



Figure 1 DMB rebar counting system with conveyor belt and sophisticated machinery.

While current automated rebar counting systems, such as the one developed by DBM, offer remarkable accuracy and efficiency, their widespread adoption faces several significant challenges (info@dbmsteel.com, 2023). These advanced systems leverage sophisticated machine vision and AI technologies to achieve an error rate as low as 5 errors per 10,000 counts and can process rebars at a rapid speed of 570 mm/s. However, the practicality of implementing such automated counting systems is limited by high capital costs, infrastructure modifications, and the need for specialized technical expertise. These systems are optimized for controlled environments and standardized rebars, limiting their effectiveness in the variable conditions commonly encountered in construction sites.

2.2.3 Classical Computer Vision for Object Detection

Early attempts at automating rebar counting leveraged traditional techniques, including edge detection and template matching. These approaches helped identify the challenges and limitations of classical methods in handling complex scenes, motivating our exploration into deep learning-based approaches.



Figure 2 A failed attempt at detecting rebars using Canny Edge Detection (OpenCV: Canny Edge Detection, n.d.)

Contour Based Detection: Techniques like the Canny Edge Detector (Canny, 1986) identify edges by detecting intensity gradients in images. While effective in simple scenarios, these methods failed to capture individual rebar heads, were extremely sensitive to lighting conditions, and did not offer a reliable counting mechanism. They also struggled with complex backgrounds.

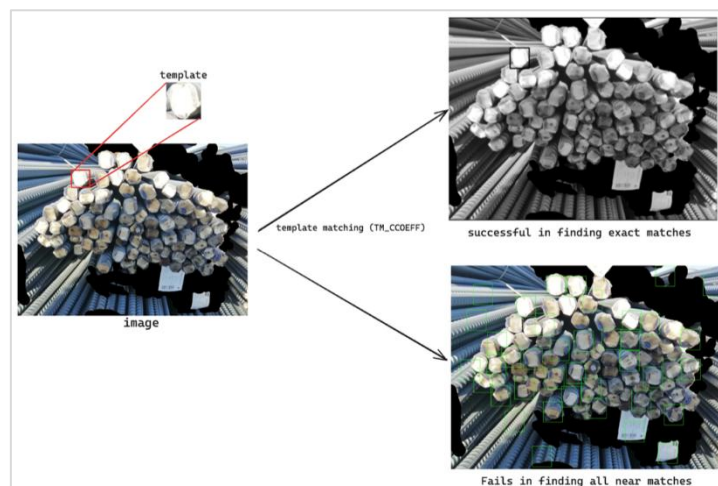


Figure 3 A failed attempt at detecting rebars using template matching (OpenCV: Template Matching, 2024)

Template Matching (*OpenCV: Template Matching*, 2024): This approach involves sliding a template image of a rebar over the target image to find matches based on similarity metrics. Template matching is sensitive to variations in rebar size, orientation, and occlusion, making it ineffective in real-world scenarios where rebars are not uniform.

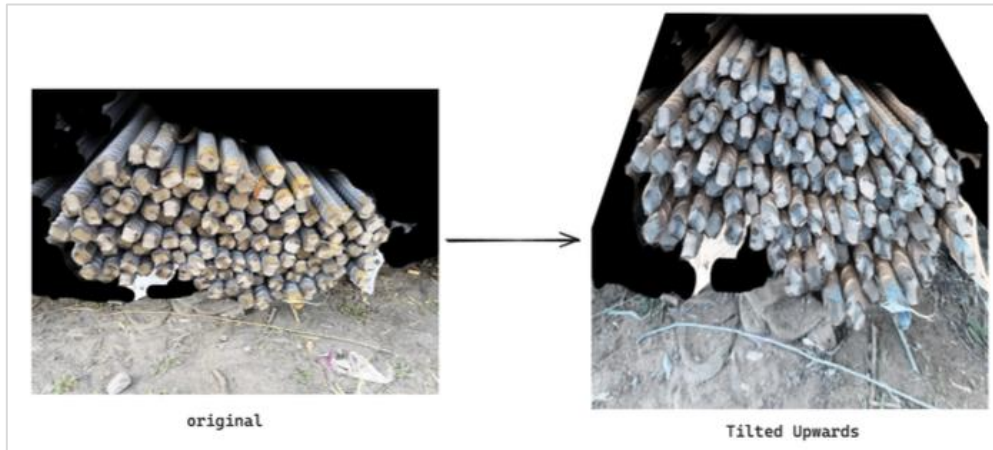


Figure 4 A failed attempt at detecting rebars using perspective transformation.

Perspective Transformation (*OpenCV: Geometric Image Transformations*, n.d.): This method involves transforming the perspective of an image to correct skewed or angled views of rebars, making them appear front-facing and aligned. However, it has notable drawbacks: it requires user input for positional points, struggles with overlapping rebars, and can cause contortions when multiple rebars are present. These limitations make classical perspective transformation unsuitable for reliable rebar detection.

2.2.4 Deep Learning for Object Detection

Deep learning has emerged as a powerful solution for automating object detection tasks due to its ability to learn complex feature representations and adapt to diverse scenarios. The field of deep learning has progressed rapidly, with a variety of architectures tailored for object detection, each with its own strengths and trade-offs. The following subsections summarize the key deep learning-based approaches relevant to rebar counting:

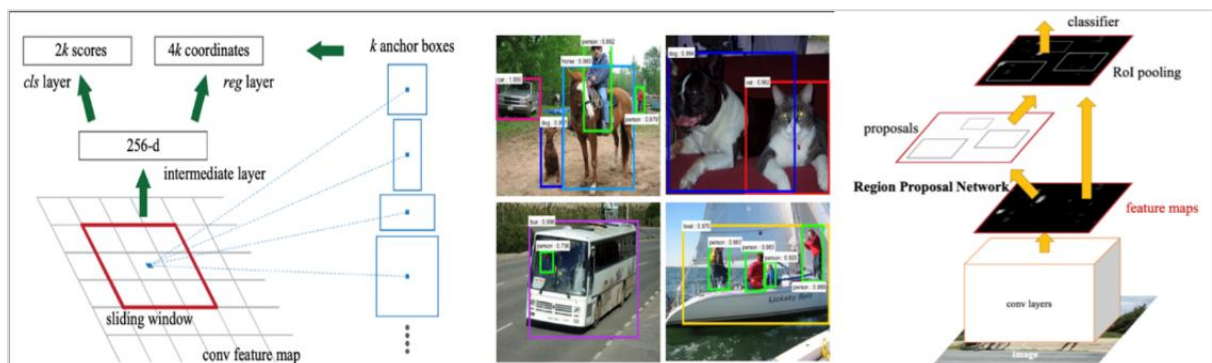


Figure 5 A RPN module that gives region proposals for where multiple objects (of different shapes) may sit

Faster R-CNN (Ren et al., 2016): Introduced a Region Proposal Network (RPN) that efficiently generates region proposals for object detection. This network enables end-to-end detector training on shared convolutional features and significantly decreases computation in predicting multiple objects of varying sizes. Faster R-CNN has achieved high accuracy on standard datasets but faces challenges in rebar counting due to the densely packed and overlapping nature of rebars.



Figure 6 Illustrates how YOLO models detection as a regression problem. It resizes the input image to 448×448 , runs a single convolutional network on the image, and thresholds the resulting detections by the model's confidence.

YOLO Series (Reis et al., 2024): YOLO models perform object detection in a single pass, offering high processing speeds suitable for real-time applications. While efficient, they may not achieve the same level of accuracy as two-stage detectors like Faster R-CNN or Mask R-CNN, particularly with small or overlapping objects. Their grid-based detection approach can lead to missed detections or inaccuracies in counting rebars.

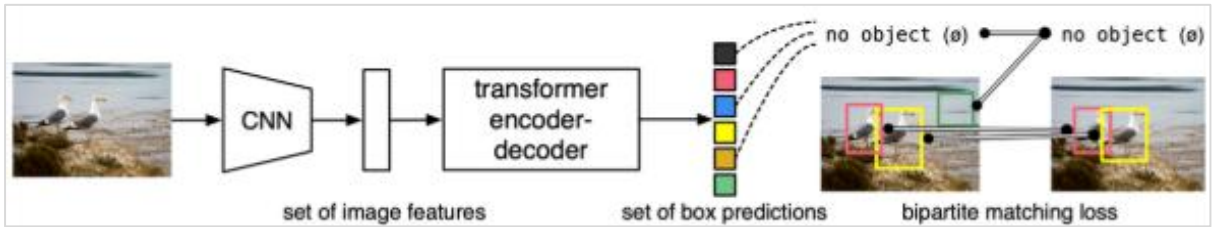


Figure 7 Object detection as a direct set prediction problem with DETR, where predictions are paired with classes of objects

Detection Transformer (DETR) (Carion et al., 2020): Combines CNNs with transformers to model global relationships in images. It simplifies the detection pipeline by removing the need for components like anchor boxes and non-maximum suppression. DETR has demonstrated strong performance on challenging datasets but requires large amounts of training data and significant computational resources. In the context of rebar counting, the scarcity of annotated data and resource constraints limit the practicality of DETR.

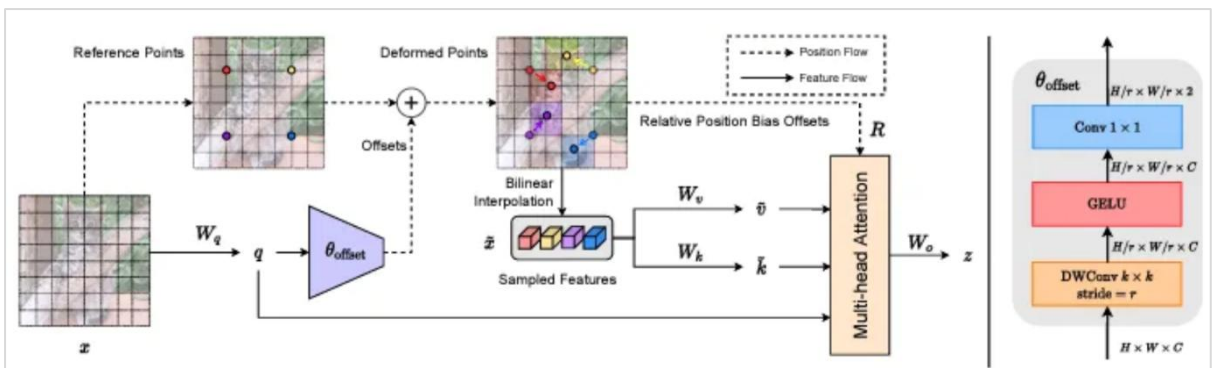


Figure 8 Deformable Attention Module

DETR-Deformable (Zhu et al., 2021): DETR-Deformable builds upon the Detection Transformer (DETR) architecture by introducing deformable attention modules, which allow the model to focus on important features within local regions rather than relying solely on global context. This design significantly improves performance on dense and complex scenes, making it particularly suitable for rebar counting tasks where rebars often overlap or are partially occluded. This architecture also provides faster convergence during training, which is particularly helpful in scenarios with scarce annotated data, such as rebar counting datasets.

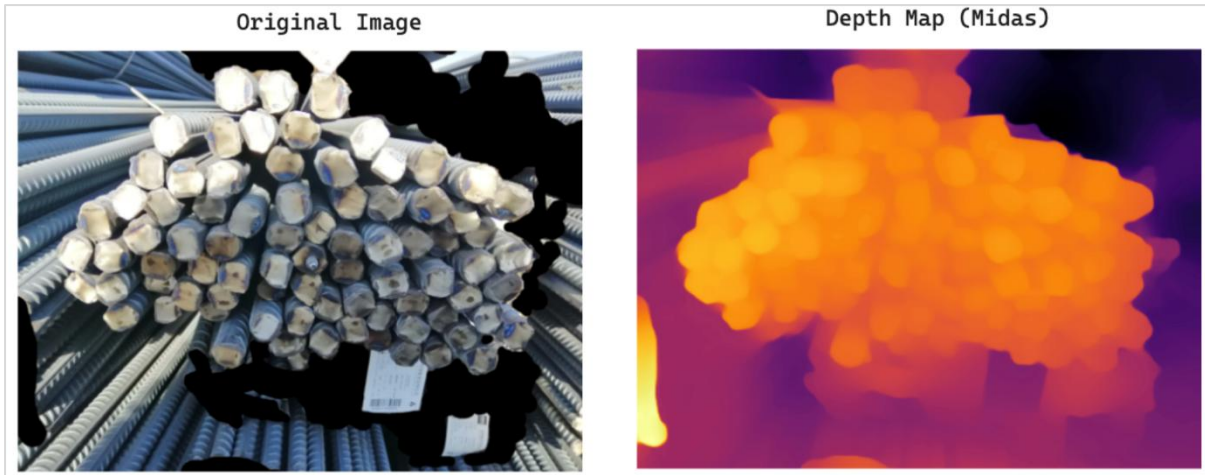


Figure 9 (Left) Original Image. (Right) Depth Map generated using MiDaS.

Depth Estimation (MiDaS) (Zhu et al., 2021): MiDaS provides depth estimation from single images, using advanced monocular depth estimation techniques. It helps to generate depth maps that can enhance detection performance in cluttered scenes where rebar overlap. This technique theoretically shows promise for improving overall detection robustness for other detection methods.

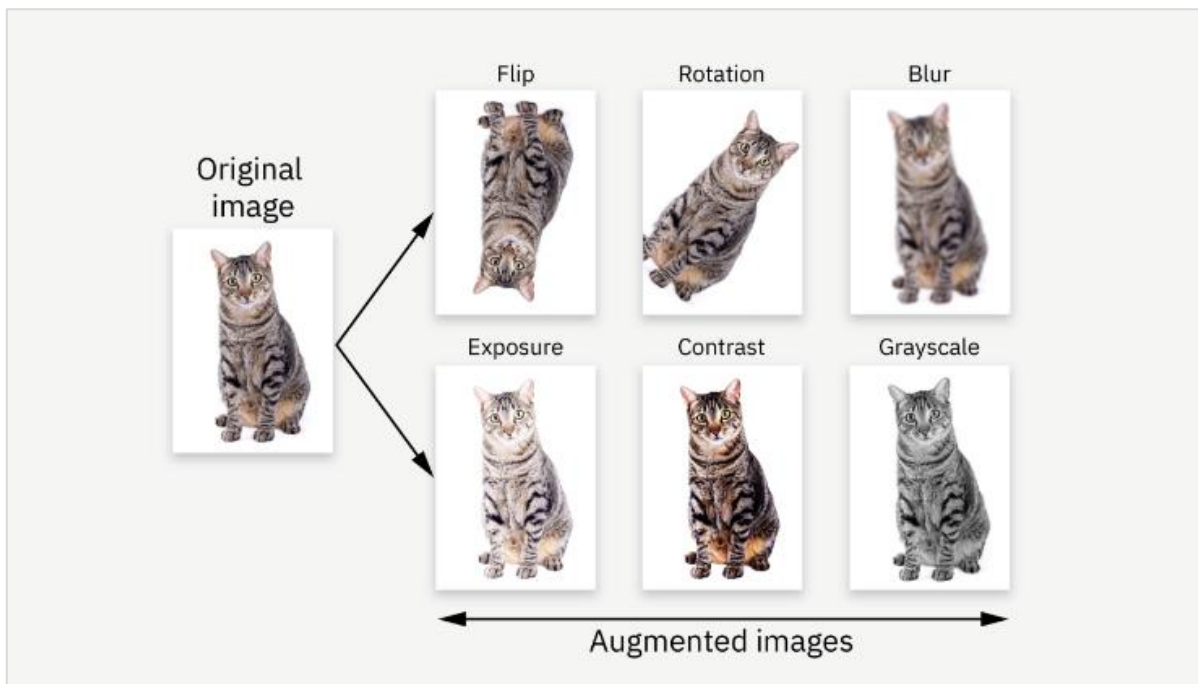


Figure 10 (Left) Original Image. (Right) artificially synthesized augmented images from the same original image.

Data Augmentations (What Is Data Augmentation?, 2024): Data augmentation is a crucial preprocessing step in deep learning that artificially increases the diversity of training data, enhancing model robustness and generalization. It artificially expands the training data by applying transformations to existing images, such as flips, rotations, crops, and color adjustments. Data augmentation helps models become robust to real-world variations in images, like changes in lighting, angles, and object arrangements.

2.3 Project Rationale and Significance

Implementing computer vision and deep learning for rebar counting offers significant benefits over traditional manual methods and high-end automated systems:

- **Efficiency and Automation:** Computer vision automates the counting process, drastically reducing time and labor. Manual counting often requires recounts to avoid errors, which quadruples the time needed to count hundreds of rebars.
- **Enhanced Accuracy:** Deep learning algorithms can detect and count rebars with high precision, minimizing human errors.
- **Cost-Effectiveness:** Compared to high-end systems, computer vision solutions use off-the-shelf hardware and open-source software, reducing development costs.
- **Flexibility and Adaptability:** Computer vision systems can adapt to various environments and handle rebars of different sizes, shapes, and conditions.
- **Robustness to Variability:** Deep learning models can handle variations in rebar appearance, lighting conditions, and cluttered backgrounds.
- **Scalability and Adaptability:** Once trained, models can process large volumes of data consistently. Fine-tuning pre-trained models enhances adaptability to new environments.

In summary, a deep learning-based solution provides a practical and scalable approach to address the industry's needs for reliable, efficient, and cost-effective rebar counting.

3. Project Outcomes

3.1 Overview of Implemented System

The delivered solution is an integrated, end-to-end system for automated rebar counting, intended to replace manual counting methods in industrial supply chain workflows. The system is designed to enable users to capture, upload, and annotate images of rebars via a mobile application, receive rapid and accurate count results, and securely store all relevant data in the cloud for future access and analysis.

3.1.1 Features and Capabilities

The following table below summarizes the primary features and capabilities of the rebar count system:

Table 1 Features and Capabilities of the Implemented System

Feature / Capability	Description
AI-Powered Rebar Counting	Utilizes a trained YOLOv8 deep learning model to detect and count rebars from user images.
Mobile Application (Android)	User-friendly app for image capture, annotation, history review, and result visualization.
Real-Time Processing and Feedback	Delivers rebar count results within approximately 2–5 seconds per image (measured on backend).
Image Annotation	Allows users to annotate captured images to mask irrelevant regions, improving counting accuracy.
Count History and Analytics	Stores and displays user-specific count history, including timestamps, annotated images, and statistics.
Secure User Authentication	Implements Firebase Authentication for account creation and secure access.
Cloud-Based Data Management	Uses Firebase Firestore and Storage to store count results and image data.
CI/CD Automated Backend Deployment	Backend is deployed on Render.com with CI/CD automation using GitHub Actions and Docker.
Cross-Platform Support (Android 8+)	Compatible with all Android devices supporting Google Mobile Services.

3.1.2 Delivered Components

The system is composed of several tightly integrated components, each developed and validated for reliable, end-to-end operation. The overview of the principal deliverables is outlined below, for a more detailed view of all project deliverables, please refer to 5. *Software Deliverables*.

Table 2 Delivered Components of the Rebar Counting System

Component	Description
Android Mobile App	Provides UI for onboarding, authentication, image capture/upload, annotation, result display, and count history. Developed using Kotlin, Jetpack Compose, and Firebase SDK.
YOLOv8 Model	Custom-trained object detection model for rebar counting. Trained on over 1200 annotated images, fine-tuned for robustness to lighting, occlusion, and background clutter.
FastAPI Backend	RESTful backend exposing a /count/ endpoint. Handles image upload, model inference, and returns annotated results. Containerized with Docker and deployed on Render.com.
Firebase Integration	Manages user authentication (Firebase Auth), stores result metadata (Firestore), and hosts image files (Firebase Storage).
CI/CD Pipeline	Automated build, test, and deployment pipeline using GitHub Actions, Docker Hub, and Render webhooks for continuous backend updates.
User Guide & Documentation	Comprehensive end-user and technical guides documenting installation, system requirements, usage, deployment, and troubleshooting steps.
Test Report	Structured testing documentation covering white-box, black-box, integration, and end-user tests, with results evidencing feature robustness and reliability.

3.2 Results, Achievements, and Requirement Satisfaction

This section presents a critical analysis of the results achieved by the rebar counting system and evaluates how the delivered solution aligns with the requirements set out during project initiation, organized by functional and non-functional criteria.

3.2.1 Functional Results and Requirement Coverage

The functional outcomes of the project reflect successful delivery of the core features as identified in the requirements specification and iteratively refined throughout the development process. Each requirement has been validated through structured integration, system, and user acceptance testing (as documented in the *User Guides and Test Report*). Below is the summary of functional results:

1. User Authentication and Profile Management

Users can register, log in, and access their individual count history securely. Authentication is managed through Firebase Authentication, and all user actions are isolated by account.

2. Image Capture, Upload, and Annotation

The Android application enables users to capture images using the device camera or select images from their gallery. An integrated annotation interface allows users to mask irrelevant regions, which improves model inference quality for complex images.

3. Automated Rebar Detection and Counting

Upon submission, each image is transmitted to the backend API where the trained YOLOv8 model processes the image and returns a count of detected rebars. Results are delivered in near real-time and are clearly presented to the user.

4. Result Storage and History Review

Each counting result, along with associated images and metadata (timestamp, user ID), is stored in Firebase Firestore and Firebase Storage. Users can access their complete rebar count history, with results organized by session.

The table below summarizes the core functional requirements, their implementation status, and the primary method of validation. Further detail and full test evidence are available in the Test Report. All high-priority functional requirements were fully met and have been evidenced through a combination of automated test suites, manual walkthroughs, and supervisor sign-off.

Table 3 Functional Requirements Coverage

Functional Requirement	Status	Validation Method
User registration and authentication	Met	System integration tests; supervisor demo
Image capture via camera or gallery	Met	Functional tests; end-user walkthrough
Image annotation prior to submission	Met	Usability tests; demonstration logs
Submission and storage of images for counting	Met	Backend integration tests; Firestore audit
Automated rebar counting using AI model	Met	Model inference tests; count accuracy checks
Real-time feedback of results (< 5 seconds per image)	Met	Timing benchmarks; system test logs
Persistent storage and retrieval of count history	Met	Database query tests; UI review

3.2.2 Non-Functional Results and Requirement Coverage

This subsection examines the non-functional outcomes of the project, focusing on performance, accuracy, robustness, security, and usability. Each non-functional requirement is addressed with supporting metrics, benchmarks, and references to the formal test report and demonstration sessions. Below are the comprehensive non-functional performance and quality results:

1. System Performance (Latency and Responsiveness)

The backend AI service returned rebar counts with an average latency of 2.4 seconds per image (measured over 100 test submissions), and a maximum observed latency of 4.6 seconds. All submissions met the project's real-time feedback requirement of under 5 seconds. The full latency performance table is included in the appendix for reference.

2. Model Accuracy and Robustness

The YOLOv8-based detection model achieved an average counting accuracy of 96.82% on the project's validation dataset, which included a diverse set of images with varying lighting, occlusion, and background clutter. The model's mean absolute error (MAE) was 4.393 rebars per image, and its performance remained robust in challenging conditions, as evidenced by structured test cases. The full table of performance metrics is provided in Appendix.

3. Robustness to Field Conditions

The model was evaluated across images exhibiting varying lighting, glare, and occlusion. While overall detection remained strong (precision and recall above 0.97), error analysis indicated a drop in exact-match accuracy under heavy overlap and extreme lighting conditions. Targeted augmentation and user annotation mode were implemented to mitigate these edge cases.

4. Security

All user data and rebar images are transmitted over HTTPS, and authentication is enforced for all app operations through Firebase Authentication. No unauthorized data access or privilege escalation was observed in manual penetration tests.

5. Usability and Accessibility

All core user tasks were completed successfully in usability walkthroughs. Minor issues identified in error messaging and navigation were addressed in the final release. Users were able to annotate, submit, and review counts with minimal training.

The table below summarizes the non-functional requirements, their implementation status, and the primary method of validation. The system met or exceeded all key non-functional requirements except for narrowly missing the 99% accuracy benchmark (achieved 96.82%). Performance is still considered robust and fit-for-purpose, with further model improvements identified as a potential future work.

Table 4 Non-Functional Requirements Coverage

Non-Functional Requirement	Status	Validation Method
Image processing and counting within 5 seconds	Met	Timing benchmarks; backend logs; user testing
Counting accuracy $\geq 99\%$ on industry-representative data	Partially Met	Model evaluation on validation set (96.82%)
Robustness to lighting, angles, and occlusions	Met	Structured scenario testing; error logs
Secure authentication and data access	Met	Firebase Auth review; penetration test
User-friendly and accessible interface	Met	Usability walkthroughs; user feedback
Annotation capability for improved accuracy	Met	Usability and model performance tests

3.2.3 Requirements Traceability Matrix

The full Requirements Traceability Matrix, including detailed validation methods and evidence, is provided in Appendix C – Requirement Traceability Matrix, which provides a comprehensive mapping of all functional and non-functional requirements against their implementation and validation status. This ensures transparent demonstration of requirement satisfaction and provides an audit trail for project assessment. All identified requirements, both functional and non-functional, were completed and validated through integration, system, and acceptance testing. Where requirements were derived or expanded during project execution (e.g., R10 – R13, added based on team workflow and operational needs), they have also been met and evidenced in the relevant test and deployment documentation.

3.3 Key Decisions and Justifications

Throughout the project, several key decisions significantly influenced the outcome and overall success of the system. This section summarizes the most important of these choices, along with their underlying rationale and supporting evidence.

1. Model Selection: YOLOv8 vs. Alternatives

Early in the project, both YOLOv8 and Faster R-CNN models were benchmarked on our annotated rebar dataset. While Faster R-CNN showed competitive accuracy, YOLOv8 consistently delivered higher inference speed and similar or better detection precision and need for deployment scalability, YOLOv8 was selected as the production model (Reis et al., 2024). This choice was further supported by YOLOv8’s robust community support and easier integration with the FastAPI backend.

2. Backend Framework and Hosting

The backend was implemented using FastAPI and deployed on Render.com. This decision was made based on FastAPI’s high performance, modern syntax, and strong support for asynchronous operations, enabling the low-latency processing required for real-time feedback. Render.com was chosen for its simple deployment workflow, support for Dockerized applications, and automatic scaling, which aligned with our needs for rapid iteration and reliable public access.

3. Mobile Stack: Kotlin/Android Studio over Flutter

The team initially explored Flutter/Dart for cross-platform development but encountered significant integration challenges and lower familiarity across members. A rapid pivot to Kotlin

and Android Studio enabled faster development, easier debugging, and native access to the required Android APIs. This decision led to more stable app releases and allowed for efficient testing across a wider range of devices.

4. Cloud vs. On-Device Inference

Inference was deployed on the cloud backend rather than locally on the device to ensure model version control, simplify updates, and offload computational demands from user devices. This approach allowed all users to benefit from the latest model improvements without requiring app updates and ensured consistent performance across a broad spectrum of Android devices.

5. Inclusion of Image Annotation Feature

The image annotation feature was prioritized after early usability testing and supervisor feedback highlighted difficulties in achieving accurate counts in images with overlapping or partially obscured rebars. By allowing users to mask irrelevant or noisy regions, annotation directly improved model performance on edge cases and was well received by users.

3.4 Comparison to Existing Solutions / State of the Art

Manual Rebar Counting (Industry Baseline)

Manual counting remains common practice in many construction supply chains. It is highly labor-intensive, subject to human error, and often leads to inconsistent inventory records, especially in scenarios involving overlapping, bundled, or partially obscured rebars. Typical error rates reported in the literature range from 5% to 10%, with significant time required for large batches. Manual processes are also challenging to audit and difficult to integrate with digital inventory systems.

Automated Solutions in Literature

Automated counting solutions using computer vision and earlier deep learning models (e.g. Faster R-CNN, YOLOv5) have shown promise but struggle with variability, clutter, and occlusion. DBM systems try to manage these by using controlled conveyor belts and specialized machinery, but this adds cost and complexity. Their approach achieves very high accuracy (5 errors per 10,000 counts at 570 mm/s), but comes with significant expenses, infrastructure needs, and technical support challenges that rendering their practicality near impossible on most sites.

Our System's Performance

The deployed system achieved a counting accuracy of 96.8% (COCO-style) with a mean absolute error of 4.4 rebars/image and end-to-end latency well below 5 seconds. This is comparable or superior to many state-of-the-art models tested in similar contexts, with additional robustness provided by the annotation tool. Unlike many academic prototypes, our solution is fully integrated with a user-friendly mobile app, cloud-based backend, and secure data management, enabling direct field use without specialized hardware or technical expertise.

Comparison Highlights

- **Accuracy:** Matches or outperforms most published automated approaches on diverse, industry-supplied test data.
- **Speed:** Capable of Real-time inference (average ~2.4 seconds per image), outperforming most academic or commercial research solutions that often require offline batch processing.
- **Usability and Deployment:** Provides an end-to-end workflow, including secure login, annotation, and history management, these features often missing from commercial demos.

- **Adaptability:** Annotation and error handling features improve robustness in practical edge cases, an area where many solutions underperform.
- **Cost-Effective:** Achieves high performance with lower deployment and infrastructure costs with no expensive machinery.

Limitations Compared to Ideal State-of-the-Art

While performance is strong, exact-match accuracy (26.7%) indicates room for improvement, particularly in highly complex images. Further gains may be possible with larger datasets, ensemble methods, or more advanced post-processing, as reported in cutting-edge research.

Due to time constraints, further research exploration on improving the YOLOv8 architecture was not completed. For example, building a custom multimodal YOLOv8 which incorporates depth estimation, which theoretically may improve performance.

In summary, the delivered system advances beyond manual counting and most available automated methods in usability, reliability, and real-time operation, while approaching but not fully matching the highest precision benchmarks reported in controlled research or industry settings where expensive machinery is used to control the detection backgrounds.

3.5 Limitations and Challenges

While the developed AI-driven rebar counting system achieved most project objectives, several limitations and challenges emerged during development, testing, and real-world validation.

1. Model Accuracy in Complex Scenarios

Although the model demonstrated high precision and recall overall, exact-match accuracy (26.7%) remained low, particularly for images with heavily overlapped or tightly bundled rebars. Counting errors most frequently arose in scenarios involving significant occlusion, glare, or extreme lighting variations. While the annotation feature helped mitigate some issues, perfect automation remains challenging for such edge cases.

2. Dataset Diversity and Size

The training dataset, though carefully curated and augmented, was limited by the number and diversity of real-world industrial images available. This affected the model's generalizability, especially for rare or unusual rebar arrangements not represented in the training data. Synthetic augmentation addressed this partially but cannot fully replace actual field diversity.

3. Device and Environmental Variability

Testing was conducted on a representative but limited range of Android devices and network conditions. The system may face performance variability across other device types, screen resolutions, or lower-bandwidth environments. Limited field access constrained broader device compatibility and stress testing.

3.6 Potential Improvements and Future Work

Building upon the achievements and lessons learned during this project, several avenues for future improvement and extension have been identified:

1. Enhanced Model Accuracy and Generalization

Future work should focus on expanding the diversity and size of the training dataset, incorporating more real-world industrial images, and exploring ensemble learning approaches.

Leveraging synthetic data and domain adaptation techniques could further improve generalizability, especially for challenging scenarios with heavy occlusion or glare.

2. Offline and Edge Deployment

Implementing on-device model inference would allow the application to function without network connectivity, improving reliability in remote or low-bandwidth environments. Lightweight model quantization and optimization for edge devices are promising directions for achieving efficient offline performance.

3. Broader Device and Platform Support

Extending compatibility to iOS devices and supporting a wider range of Android hardware would expand the system's usability in the field. Additional accessibility features and language localization could further enhance adoption across different user groups.

4. Methodology

4.1 System Design and Architecture

4.1.1 Final System Architecture

The final system architecture of the AI-driven rebar counting solution employs a modular, cloud-first approach that balances real-time performance, scalability, and maintainability. As illustrated in Figure 1, the system consists of four major domains: the mobile client, Firebase cloud services, the backend inference API, and the continuous integration and deployment (CI/CD) pipeline.

At the core of the workflow, the mobile app enables users to capture or select images, annotate as needed, and trigger the rebar counting process. User authentication and secure data storage are managed by Firebase Authentication, Firestore, and Storage. The app interacts with a FastAPI backend deployed on Render.com, which is responsible for running the YOLOv8 object detection model and returning the detected rebar count. The backend and mobile components are kept up-to-date and reproducible through a CI/CD pipeline leveraging GitHub Actions, Docker Hub, and automated deployments to Render.com. All components interact via secure REST APIs or cloud service SDKs, ensuring a seamless flow from image capture to count retrieval and historical record management.

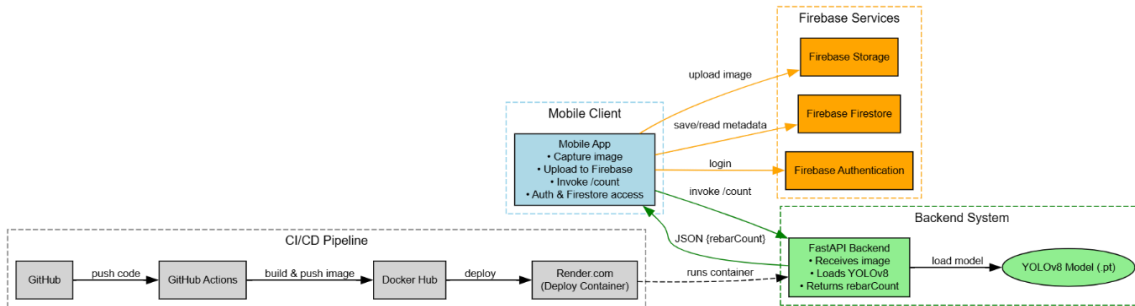


Figure 11 System Architecture Overview

Within the mobile client, the application follows a **Model-View-Controller (MVC)** architectural pattern. The Model comprises the data classes and Firebase services managing user records and annotations. The View is implemented using Jetpack Compose to render the UI and interact with users. The Controller logic, such as input handling, navigation, and triggering backend API calls, is managed by ViewModels and composable event handlers. This internal structuring enhances code readability, maintainability, and facilitates isolated testing of each layer.

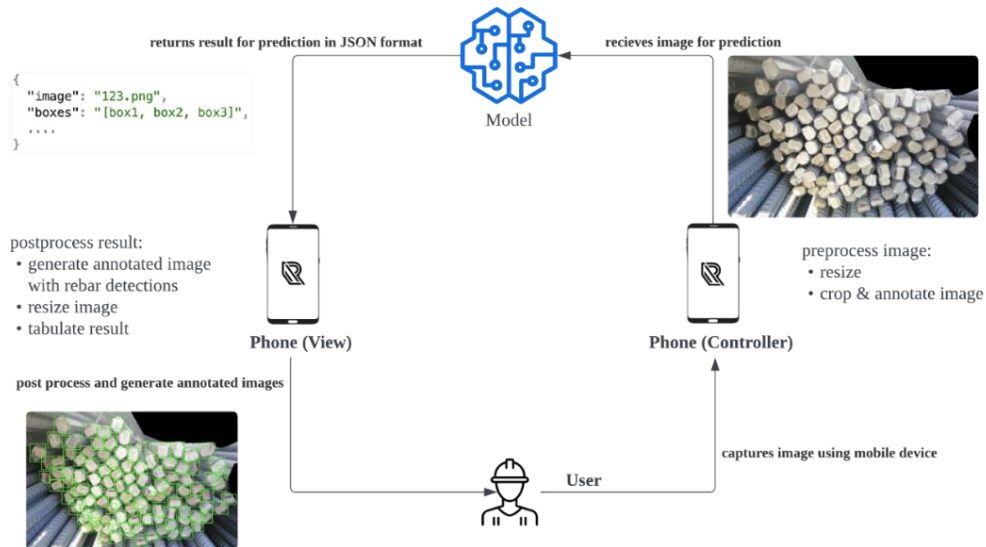


Figure 12 Model View Controller (MVC)

4.1.2 Component Breakdown

Each core system component is designed with a focused responsibility, supporting clear separation of concerns and easy extensibility. The major components are:

Mobile Application (Android, Kotlin)

- **Responsibilities:**
 - Capture images via camera or select from gallery
 - Annotate images to mask irrelevant regions
 - Invoke backend inference for rebar counting
 - Display results and history to the user
 - Manage secure authentication via Firebase
- **Key Technologies:** Android Studio, Kotlin, Jetpack Compose, Retrofit

Firebase Cloud Services

- **Firebase Authentication:** Handles user registration, login, and secure session management
- **Firebase Firestore:** Stores user profiles, count results, and metadata for each session
- **Firebase Storage:** Persists original and annotated image files for traceability and audit

Backend Inference API (FastAPI, Render.com)

- **Responsibilities:**
 - Receives images from the mobile app via RESTful endpoint
 - Loads the YOLOv8 model and performs inference
 - Returns rebar count and base64-encoded rebar image with bounding boxes
 - Ensures scalability and secure access
- **Key Technologies:** FastAPI, Python, Docker, Render.com

YOLOv8 Model (.pt):

- **Responsibilities:**
 - Detects and counts rebars in uploaded images
 - Trained on a custom, annotated dataset to optimize real-world performance
- **Key Technologies:** PyTorch, Ultralytics YOLOv8

CI/CD Pipeline

- **Responsibilities:**
 - Automates backend testing, building, and deployment
 - Ensures that code changes are reliably promoted to production
- **Key Technologies:** GitHub, GitHub Actions, Docker Hub, Render.com

This architecture and component arrangement enables secure, responsive, and scalable rebar counting that is easily maintainable and can be adapted for future enhancements.

4.1.3 Data Flow and Integration

The end-to-end data flow in the rebar counting system is designed for efficiency, data consistency, and real-time feedback. This section outlines the step-by-step process by which user actions and system components interact to deliver seamless rebar counting and historical tracking.

End-to-End Workflow

1. **User Authentication:**
 - The user opens the mobile application and authenticates via Firebase Authentication. Only authenticated users can proceed to image capture and counting workflows.
2. **Image Capture and Annotation:**
 - The user captures a new photo of rebars or selects an existing image from the device gallery.
 - If needed, the user can annotate the image within the app to mask out irrelevant or noisy regions, improving model accuracy for complex cases.
3. **Image Upload and Backend Invocation:**
 - The annotated (or original) image is uploaded to Firebase Storage for secure and persistent storage.
 - Simultaneously, the image is transmitted via a RESTful API call to the FastAPI backend for inference.
4. **Backend Inference:**
 - The FastAPI backend receives the image, loads the YOLOv8 model, and performs object detection to count the number of rebars present.
 - The backend returns the detected count, along with base64-encoded rebar image bounding boxes drawn to the mobile app.
5. **Result Storage and User Feedback:**
 - The rebar count result, along with image metadata (timestamp, user ID, optional location), is written to Firebase Firestore for historical record-keeping.
 - The user is notified of the result within the mobile app, and the result is immediately reflected in their count history.
6. **Historical Record Management:**
 - The user can view previous count results, associated images, and details at any time via the app's history screen, with data fetched in real-time from Firestore and Storage.

Integration Points

- All communication between the mobile app, backend, and Firebase services is secured using industry-standard authentication and encrypted channels (HTTPS).
- The system architecture supports horizontal scaling; backend instances can be replicated via Render.com, and Firebase’s managed infrastructure handles increased data storage and retrieval demands.
- The CI/CD pipeline ensures backend updates are seamlessly integrated, maintaining consistent API contracts and minimizing downtime.

Rationale and Benefits

- This architecture supports near real-time feedback, robust data traceability, and future extensibility (e.g., additional analytics, offline mode, or new user roles).
- Decoupling inference from data storage and authentication allows for independent scaling and targeted performance optimizations.

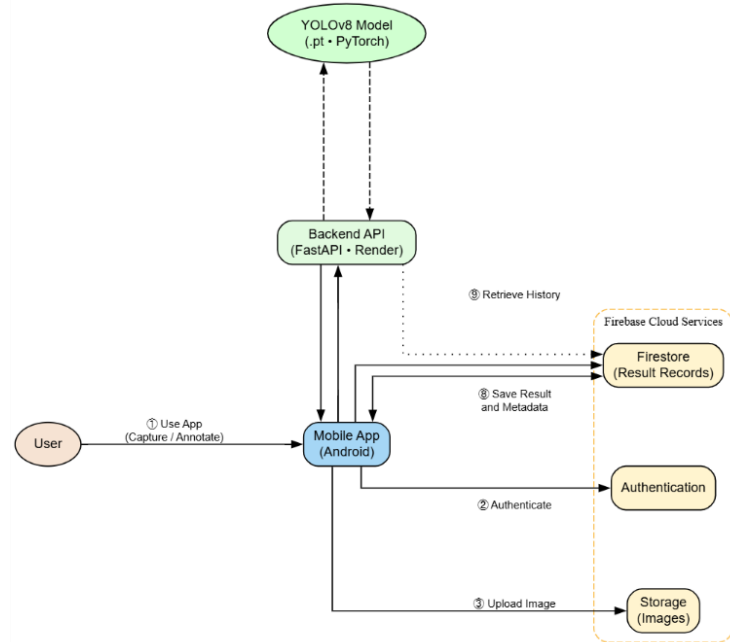


Figure 13 Data Flow Diagram

4.2 Software Tools and Technologies

This section outlines the comprehensive set of tools, libraries, platforms, and hardware used to build, test, deploy, and maintain the AI-powered rebar counting system. The choices reflect a combination of performance, compatibility, and familiarity with the development team’s skillset.

Programming Languages, Frameworks, and IDEs

Table 5 Programming Languages, Frameworks, and Development Environments

Category	Technology	Purpose
Programming Languages	Python, Kotlin	Python for backend and model; Kotlin for Android app
AI Frameworks	PyTorch, Ultralytics YOLOv8	Deep learning model training and inference
Data Annotation	Roboflow	Curate, label, merge, and augment rebar image datasets
Web Framework	FastAPI	Backend API development
Mobile UI Toolkit	Jetpack Compose	Declarative UI development for Android
IDEs	Android Studio, Visual Studio Code	Kotlin and Python development environments
Cloud IDE	Google Colab (Jupyter)	Model training and experimentation

Core Libraries, SDKs, and Dependencies

Table 6 Key Libraries and SDKs

Component	Library / Tool	Functionality
AI Model	OpenCV, NumPy, Pandas, Matplotlib	Image pre-processing and evaluation support
Data Annotation	Roboflow	Dataset annotation, augmentation, and export
Mobile App	Firebase SDKs, Retrofit, Coil	Authentication, REST API integration, image loading
Backend API	Uvicorn, Pydantic	ASGI server and data validation in FastAPI
Version Control	Git, GitHub	Repository management and collaboration
CI/CD	GitHub Actions	Automates Docker builds, testing, and deployment
Testing Tools	Android Emulator, curl, Swagger UI	API and app validation in development

Cloud Services and Deployment Infrastructure

Table 7 Cloud Infrastructure Tools

Platform / Tool	Purpose
Firebase Authentication	Secure user login and session control
Firebase Firestore	Cloud-hosted NoSQL database for storing metadata and results
Firebase Storage	Upload and retrieval of image files
Render.com	Containerized hosting of FastAPI backend
Docker Hub	Registry for storing and deploying Docker images

Hardware Requirements

Table 8 Hardware Specification

Category	Specification	Purpose
Local Development	Intel i7 / Ryzen 7, 16 GB RAM, SSD, GTX 1650+	IDE usage, backend API, basic model inference
Mobile Devices	Android 8.0+, 3 GB RAM, 12 MP camera	App capture, testing, and usability validation
Network Requirement	100 Mbps+ broadband	For image transfer, inference response, CI/CD

These specifications ensured all components from data collection and model development to app usage and backend deployment could be performed efficiently, both locally and in the cloud. The adoption of a cloud-based deployment model for the backend and data storage leverages industry best practices for scalability, availability, and operational efficiency (Armbrust et al., 2010).

4.3 Model Development and Implementation

4.3.1 Data Collection

Table 9 Size of datasets collected

Dataset	Size (no. of images)	State
Externally Sourced	250	Annotated
Company	95	Non-Annotated (Raw Image)

These datasets were collected to support robust model development. Due to the lack of annotated data from the company dataset, an external dataset was also acquired to aid in model training and validation. This ensured a comprehensive and reliable approach to developing the rebar counting system.

4.3.2 Semi-Automated Dataset Annotation

To expedite the annotation of all rebars, a semi-automated pipeline was developed. An annotator model was initially trained using an externally sourced annotated rebar dataset. The Faster R-CNN model was selected for this purpose due to its high accuracy in predictions. Subsequently, predictions were generated by running inference on the raw company dataset. Finally, these predictions were formatted as a standard COCO dataset, submitted to Roboflow, and manually inspected. Any errors made by the model were corrected to obtain the final annotated company dataset.

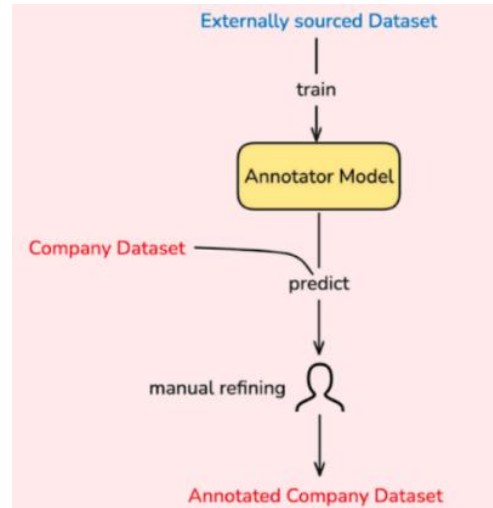


Figure 14 Semi-automated annotation pipeline

Table 10 Annotation Strategy

Annotation Strategy	Type
Format	COCO
Classes	["rebar"]
Class Labels	["background", "rebar"]
Annotation	Bounding boxes

This semi-automated approach significantly reduced the time required to annotate the entire dataset, streamlining the overall development process.

4.3.3 Initial Model Training and Selection

Dataset Split Strategy

Due to a limited number of training samples, the validation split served as both the validation and testing split. A train-valid split of 80%:20% was used for all datasets.

Sequential vs Merged Fine-Tuning

Two identical Faster R-CNN models and YOLOv8 models were trained using two different fine-tuning approaches to investigate which approach performed the best.

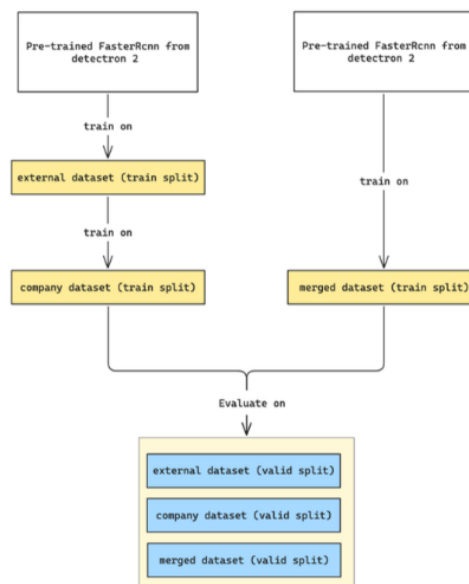


Figure 15 Sequential and Merged Fine-tuning Approaches.

It was observed that models fine-tuned on the merged dataset consistently outperformed models fine-tuned sequentially, even across different model architectures. This is attributed to the dataset variation, which enhances model generalization when trained using stochastic gradient descent, as it stochastically trains on images from both datasets simultaneously. Consequently, the final model (YOLOv8) used for the application was trained and evaluated on the merged data.

Model Comparison

We systematically experimented with different deep learning architectures and techniques to assess their suitability for rebar counting tasks using the merged dataset. The goal was to select a model that could effectively balance accuracy, speed, and deployment practicality. Each model's strengths and weaknesses for rebar detection were evaluated to understand its real-world applicability.

Table 11 Summary of the findings of each model during the selection process and its viability as a candidate detection model

Platform / Tool	Strengths	Weakness	Viability
Faster-Rcnn	Performs slightly better than YOLOv8 in terms of accuracy.	- Detectron2 framework did not support Windows, hence it was extremely hard to integrate. - Large Model Size (25MB) - Fast inference time (10 secs)	Yes
YOLOv8	- Easy integration with FastAPI. - Small model size (22MB). - Fast inference time (< 5 secs). - Possible offline mobile-only inference.	Slightly performs worse than Faster R-CNN, especially on non-rebar images; however, it is sufficient for the project.	Yes
MiDaS	No training needed for Depth Map prediction.	- Depth Map does not differentiate between background and rebar. - No direct and substantial benefits to detecting rebars. - Custom external architecture needed to provide predictions.	No
Deform-Detr, Detr	- Performs better on extreme lighting conditions given enough training, due to the attention mechanism. - Potential better generalization, given sufficient training.	- Extremely slow convergence. - Very flat (near-zero) gradients, which shows layers barely learn. - Underfitting and early convergence. - Severely underperforms in dense and large bundles. - Expensive training, which halted further progress. - Large Model Size (500MB)	No

After extensive evaluation, YOLOv8 was selected as the final model because it best met the project's requirements for integration ease, fast inference, and suitability for real-time and mobile deployment. Faster-RCNN was selected as the annotator model for the semi-automated dataset annotation pipeline, where its slight accuracy advantage was valuable despite integration challenges.

4.3.4 Model Architecture

The YOLOv8 model is designed for fast, accurate object detection in real-world scenes, which is ideal for counting rebars in construction images. It consists of four main components: Backbone, Feature Pyramid Network (FPN), Feature Pyramid, and Head. Below is a breakdown of each main component of the architecture:

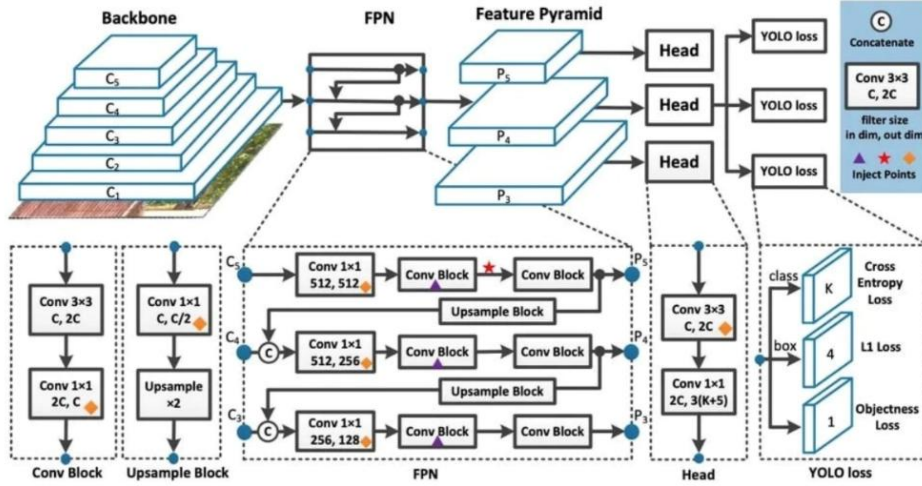


Figure 16 Architecture of YOLOv8

Backbone

The backbone is the feature extractor of the model. It processes the input image and extracts hierarchical feature maps at different levels (C_1 to C_5). Each level captures increasingly abstract representations of the input, enabling the model to detect objects at multiple scales. The backbone typically uses stacked convolutional blocks (Conv 3×3 and 1×1) to reduce spatial dimensions and increase channel depth while preserving important features.

- **Conv Block:** Stacked 3×3 and 1×1 convolutions with increasing channel dimensions (C , $2C$) to extract features and reduce computation.
- **Upsample Block:** Increases spatial resolution, useful for preserving fine details during feature fusion.

Feature Pyramid Network (FPN)

The FPN is responsible for aggregating features from different backbone levels. By combining high-level semantic information from deeper layers with low-level detail from shallower layers, it improves detection of objects at various scales, essential for accurately identifying small or overlapping rebars.

- **Conv 1×1 :** Reduces the number of channels while retaining essential information.
- **Feature Fusion:** Combines features from multiple backbone layers, enabling multi-scale object detection.
- **Upsample Block:** Used to align feature map sizes for effective fusion.

Feature Pyramid

After the FPN fusion, YOLOv8 produces a set of three feature maps, commonly called P_3 , P_4 , and P_5 with each covering a different scale:

- **P_3** (fine resolution) $\rightarrow$ best for small objects (e.g., thin rebar ends).
- **P_4** (medium resolution) $\rightarrow$ useful for mid-sized rebars or partial views.
- **P_5** (coarser resolution) $\rightarrow$ captures larger clusters or rebars farther from camera.

Each of these maps is passed to the detection head so that rebars of varying apparent sizes can be detected accurately.

Head

The head processes multi-scale feature maps and performs the final object detection by predicting bounding boxes, class probabilities, and objectness scores.

- Conv 3×3: Extracts local features from each feature map level.
- YOLO Loss Branches: Splits into separate branches to predict:
 - Class Scores: Uses Cross Entropy Loss to classify objects.
 - Bounding Boxes: Uses L1 Loss to regress box coordinates.
 - Objectness Score: Uses Objectness Loss to determine object presence.

Because YOLOv8 is **anchor-free**, it does not rely on predefined anchor boxes. Instead, it directly predicts object centers and sizes, simplifying training and improving detection of densely packed rebars.

The YOLOv8 architecture features a simple yet powerful design that ensures fast inference times by avoiding computationally heavy attention layers. Its straightforward convolutional blocks, efficient feature pyramid integration, and anchor-free head allow it to process images rapidly and deliver accurate detections, even in challenging, real-world conditions. This makes it highly suitable for practical deployment in the construction industry, where both speed and accuracy are essential.

4.4 Implementation Activities and Process

4.4.1 Development Workflow

The development process followed a structured, component-oriented approach that allowed the system to be built, integrated, and validated in a modular and repeatable manner.

Mobile Application Development

- Implemented using **Android Studio (Kotlin)** and Jetpack Compose.
- Initial setup included Firebase integration for **Authentication, Firestore, and Storage**, followed by construction of key UI components: login/register screens, image capture, gallery selection, image annotation, result display, and history listing.
- Retrofit was used to configure HTTP communication with the backend.
- Firebase SDKs were integrated to handle real-time database writes, image uploads, and retrieval operations.

Backend API Development

- Built using FastAPI and containerized with Docker for reproducibility and deployment.
- A /count endpoint was created to receive images in base64 format, decode and preprocess them using OpenCV, and invoke the YOLOv8 model for inference.
- The API handled return of both the predicted count and bounding boxes drawn rebar image.
- The backend was deployed using Render.com, with automated builds triggered via GitHub Actions and Docker Hub.

Model Development

- Dataset preparation was conducted using **Roboflow**, which handled annotation and augmentation (e.g., brightness variation, flips, scaling).
- Model Experimentation and Selection are trained and done within lightning.ai using PyTorch and other Deep Learning Frameworks like Detectron2 for Faster-Rcnn.
- YOLOv8 model was trained in **Google Colab**, using PyTorch and the Ultralytics framework.
- Training and validation steps, along with hyperparameters, dataset formats, and result tracking, are fully documented in the Technical Guide.
- The trained .pt model was exported and loaded directly into the backend service.

Integration and Testing Pipeline

- After completing module development, the mobile app, backend, and model were integrated and tested using both physical devices and emulators.
- Detailed configuration steps for each service (Firebase, Android permissions, API URL settings, Docker setup) are included in the *Technical Guide* to ensure repeatability.
- Edge cases, such as invalid image input or empty submissions, were handled using client-side and backend-side validation logic.

Repeatability Guarantee: To ensure the solution can be reproduced by other developers, the complete technical setup, including environment requirements, installation steps, backend API schema, model loading procedures, and mobile app integration is documented in the *Technical Guide and Test Report*. A competent developer following these instructions can confidently recreate the full end-to-end system with the same outcome.

4.4.2 Testing and Validation Approach

Testing and validation were conducted in structured layers to ensure that each system component met its intended functional and non-functional requirements. The testing strategy was designed to verify correctness, robustness, performance, and usability across both the mobile and backend systems.

Unit Testing

Each component was individually validated at the function or module level:

- The mobile app's input handlers, activity flows, and annotation tools were tested using Android emulator tools and logging assertions.
- Backend utility functions for image decoding, validation, model inference, and error handling were verified with controlled inputs.
- Pydantic schemas ensured robust API request/response validation.

Integration Testing

Mobile and backend components were integrated and tested to ensure seamless data exchange and system consistency:

- API endpoints were tested using both the mobile app and tools such as curl and **Swagger UI** to simulate various input scenarios (valid/invalid base64 images, large images, missing fields).
- Image upload and retrieval via **Firebase Storage**, and metadata handling via **Firestore**, were verified by confirming data persistence and retrievability for each user session.

System Testing

Full end-to-end workflows were tested on both physical Android devices and emulators to simulate real-world usage:

- A user would authenticate, capture or select an image, annotate it, and submit it for counting.
- The app would receive the count result and update the user's history view.
- System behavior was tested across different lighting conditions and image types.
- Timing tests confirmed that inference latency consistently remained below 5-second threshold.

Model Evaluation and Validation

The YOLOv8 model was evaluated on a held-out validation set of 135 annotated rebar images:

- Metrics such as precision (0.983), recall (0.971), mAP@0.5 (0.988) and mean absolute error (4.39 rebars) were recorded.
- Scenario-based testing (e.g., occlusion, glare, bundled rebars) was performed to assess the model's robustness.
- Annotated output images and metric summaries are provided in the test report.

User Acceptance Testing (UAT)

Final testing included walkthroughs with project team members and supervisor observations:

- Users evaluated the app's usability, result clarity, annotation ease, and workflow completeness.
- Feedback was used to refine UI components, improve error messaging, and simplify image submission flows.

Test Traceability and Repeatability: All test cases, expected outcomes, actual results, and test environments are logged and traceable via the **Test Report**. Developers or assessors may replicate tests using the same annotated images and steps outlined in the document to verify consistent performance.

4.5 Deviation from Initial Design

While the overall system architecture and project direction remained aligned with the original proposal, several deviations were made during implementation to accommodate technical constraints, usability considerations, and feedback from testing. These deviations were necessary to improve system reliability, development efficiency, and user experience.

Model Selection

The initial proposal model selected was the state-of-the-art transformer model Deformable-DETR. This was due to it having the best state-of-the-art performance on large public datasets. However, during the selection process, the model faces numerous challenges while training such as vanishing gradients and slow convergence, which led to our decision to use YOLOv8 as the primary model for detection.

Mobile App Framework Change

The initial proposal outlined the use of **Flutter** for mobile development to support cross-platform compatibility. However, early development encountered significant integration challenges with Firebase SDKs and image annotation features in Flutter. As a result, the team pivoted to **native Android development using Kotlin and Jetpack Compose**, which allowed for better performance, more precise control of UI behavior, and faster development iteration.

Cloud Inference vs. On-Device Processing

The proposal considered the possibility of performing inference directly on the mobile device. However, due to model size, performance limitations on mid-range Android hardware, and deployment complexity, the team opted for a **cloud-based inference approach using a FastAPI backend hosted on Render.com**. This ensured consistent performance across devices and enabled backend-level logging and model version control.

Introduction of Annotation Tool

The original design did not include manual image annotation on the client side. After early testing revealed difficulties in model accuracy on cluttered or occluded images, an **annotation tool was introduced within the app**, allowing users to mask out irrelevant regions prior to submission. This addition significantly improved model accuracy on edge-case scenarios and was well-received in usability testing.

Expansion of Technical Stack for Automation and Deployment

While basic cloud deployment was mentioned in the proposal, the final implementation included a more advanced DevOps pipeline than originally planned. The use of **GitHub Actions** and **Docker Hub** enabled automated backend deployment to Render.com, improving development turnaround time and release consistency. Containerization using Docker provides a reproducible and scalable environment for backend deployment (Merkel, 2014).

Additional Non-Functional Improvements

The final implementation placed greater emphasis on system resilience and error handling than initially scoped. Features such as client-side input validation, informative error messaging, retry logic, and Firebase-based fallback storage for images were added to improve system reliability, particularly in low-connectivity environments.

These changes reflect the project team's responsiveness to real-world challenges and feedback, resulting in a solution that remained faithful to the project's core objectives while better addressing technical feasibility and end-user needs.

5. Software Deliverables

5.1 Deliverables Overview

This project produced a set of integrated software components and comprehensive documentation to deliver a robust, end-to-end AI-powered rebar counting solution. The primary deliverables, their roles, and supporting materials are summarised in the following table.

Table 12 Summary of Project Deliverables

Deliverable	Description & Key Features	Purpose & Reference
Mobile Application (APK)	Native Android app (Jetpack Compose); supports image capture, annotation, AI inference, result display, and history tracking.	User-facing data input, results, and workflows. See User Guide, Technical Guide.
Backend API Service	FastAPI-based server deployed on the cloud; provides RESTful endpoints for image upload and inference; integrates pre-trained YOLOv8 model and handles result processing and return.	Real-time AI inference and data management. See Technical Guide.
Trained YOLOv8 Model	Custom-trained object detection model (.pt file) with training scripts, inference pipeline, and performance evaluation tools included for reproducibility and potential future re-training.	Automated rebar detection and counting. See Technical Guide.
Cloud Integration (Firebase)	Firebase Authentication, Firestore, and Storage configured for secure login, data storage, and image management across all system modules.	Secure user management and persistent data storage. See Technical Guide.
Source Code	Complete, well-documented source for the mobile app, backend, model scripts, and deployment configurations (e.g., Dockerfile, requirements.txt).	Enables reproducibility and further development. See Technical Guide, GitHub Repos.
Documentation	User Guide (installation, navigation, feature overview), Technical Guide (deployment, backend/API setup), Test Report (validation and results), Project Management Report (team/process).	Full guidance for users, developers, and evaluators. All documents in project files/appendices.

All deliverables have been tested for functional stability, security, and usability. Comprehensive guides, including installation and deployment instructions, as well as validation results, are provided in the **User Guide**, **Technical Guide**, **Test Report**, and **Project Management Report**. These documents ensure that

end-users, developers, and academic evaluators can deploy, operate, and assess the system with confidence. For complete step-by-step instructions, walkthroughs, and evaluation evidence, please refer to the relevant guides and appendices accompanying this submission.

5.2 Visual Overview of System and Features

5.2.1 Mobile Application

The mobile application serves as the user-facing entry point for the entire rebar counting system, providing seamless workflows for image capture, annotation, result review, and historical tracking. The following are some key visual overviews of the mobile application:

Authentication Screen & Dashboard

The Login screen features a clean and accessible layout, designed to provide a straightforward entry point into the app. With its minimalist interface and clear input fields, it supports a focused and user-friendly sign-in experience.

With its intuitive layout, clearly labelled icons, and visually distinct action buttons, the Dashboard is designed for efficient navigation and a professional user experience. It serves as the central hub, visually guiding users to key features such as image capture, upload, and history access.

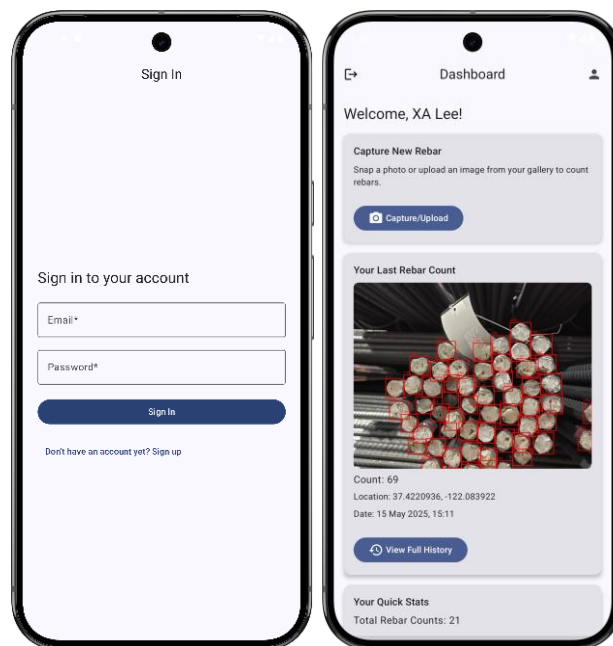


Figure 17 (Left) Login Screen, (Right) Dashboard

Image Preview & Annotate Mode

This sequence shows the app's image preview and annotation flow. It starts with the Image Preview Screen, where users can view the captured or uploaded image and confirm it.

After confirming, the Confirmation Screen allows users to either start annotating the image or send it to the AI model for processing. In Annotation Mode, users can draw on the image using Brush and Eraser tools with adjustable sizes, all kept within the image area for accuracy. Key actions like Brush, Eraser, Clear, and Save are easy to access, supporting a smooth and focused annotation experience.

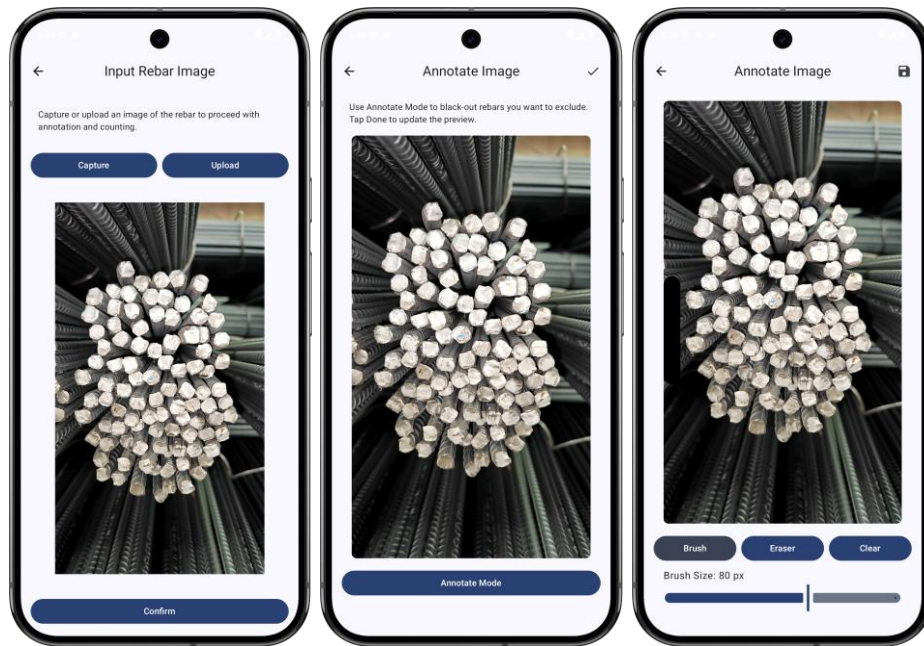


Figure 18 (Left) Image Preview, (Middle) Image Confirmation, (Right) Annotation Screen

Result Output & View Full History

The left screen presents detection results in a clean card format, showing key details like rebar count, timestamp, user ID, and location, along with a rebar image with bounding boxes. A Google Map view with a red marker adds spatial context, and tapping the image allows users to view it in full size for closer inspection.

The right screen provides a visual summary of past detections. A line graph at the top shows trends over time with options to filter by day or month. Below, each detection is listed in a card format with key details, allowing users to explore both overall patterns and individual records with ease.

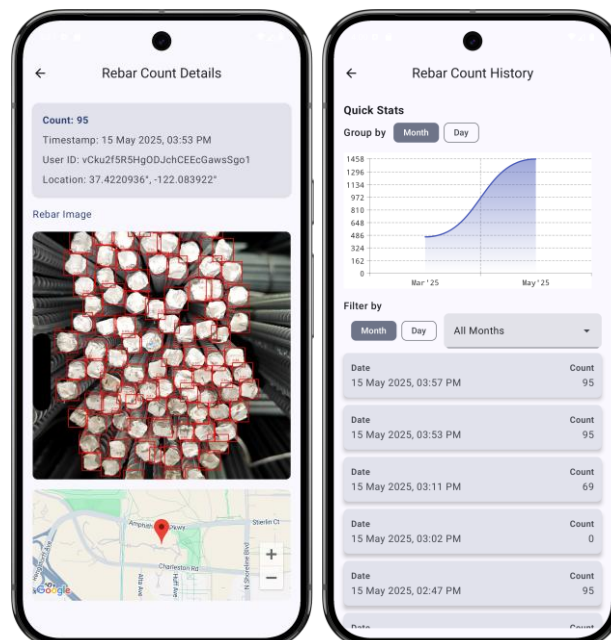


Figure 19 (Left) Rebar Detection Result, (Right) Full Rebar History Screen

For a comprehensive, step-by-step guide, including installation procedures, account setup, feature walkthroughs, and troubleshooting, please refer to the full **User Guide**, which documents every aspect of the app's operation in detail.

5.2.2 Model

The model serves as the detection backbone of the system, responsible for accurately identifying and counting rebars in uploaded images. Leveraging the YOLOv8 architecture, the model balances speed and accuracy, essential for real-time use cases in construction environments.

Inference Output

The model processes images to produce:

- Annotated bounding boxes around detected rebars with confidence scores
- Per-image rebar counts by simply counting the number of bounding boxes (done within backend)

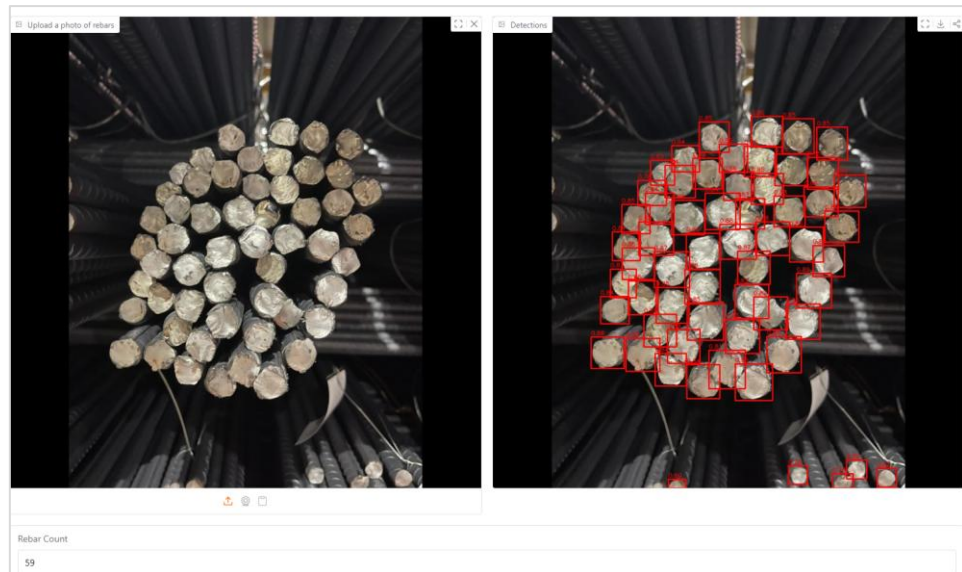


Figure 20 Inference on an image with Rebars containing bounding boxes and confidence scores

Model Weight File

The final trained model is exported as a .pt file, which is served via the FastAPI backend. This setup ensures that inference can be triggered remotely, and results can be efficiently returned to the mobile application in real-time.

For a comprehensive overview of the training and evaluation metrics, including detailed tables and discussions of test cases, please refer to the **Technical Guide**.

5.2.3 Backend and Cloud Infrastructure

The backend and cloud infrastructure form the core of the system's processing, security, and data management capabilities. This component is responsible for receiving image data from the mobile app, running AI inference, managing user authentication, and storing results in the cloud for secure access and retrieval.

Backend API and Model Inference

The FastAPI-based backend exposes a `/count` endpoint that receives images, invokes the YOLOv8 model for inference, and returns annotated results to the client. The backend is containerized with Docker for consistency and is deployed to the cloud using Render.com, which ensures scalability and robust uptime. The backend API is publicly accessible via: <https://rebar-count.onrender.com/>

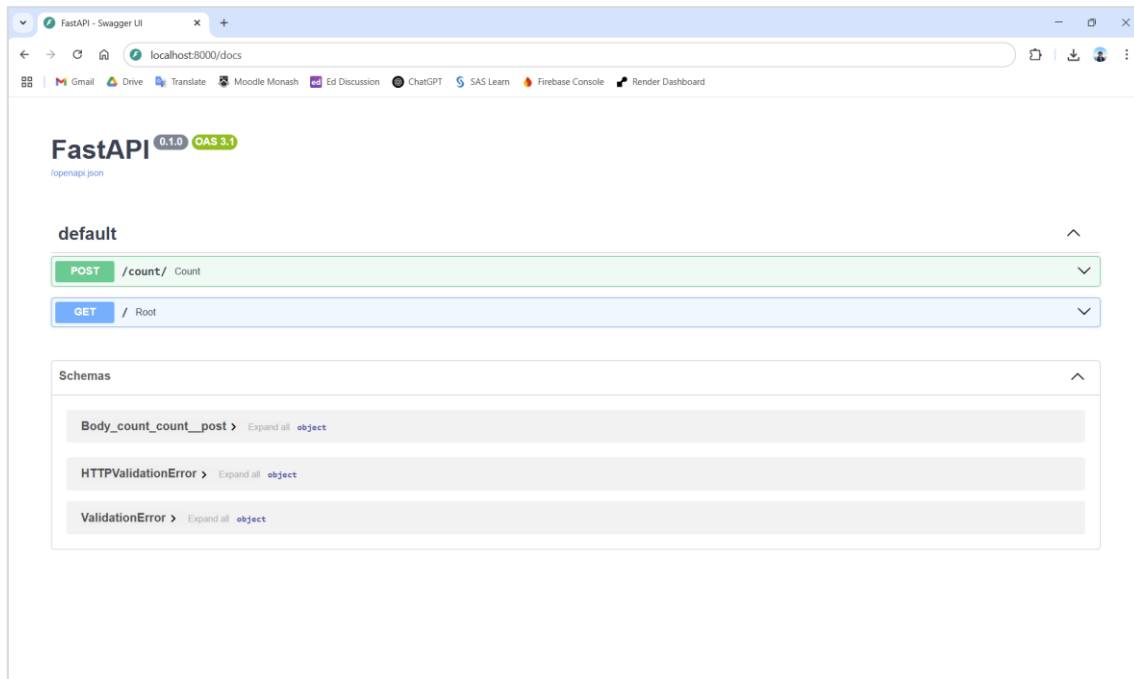


Figure 21 FastAPI Swagger/OpenAPI Documentation Page

Cloud Hosting and Deployment

Deployment is fully automated via GitHub Actions and Docker Hub, with Render.com managing container orchestration. This setup allows for rapid and reliable updates with minimal downtime, and provides a user-friendly dashboard for monitoring deployments. ([Render.com Project Link](#))

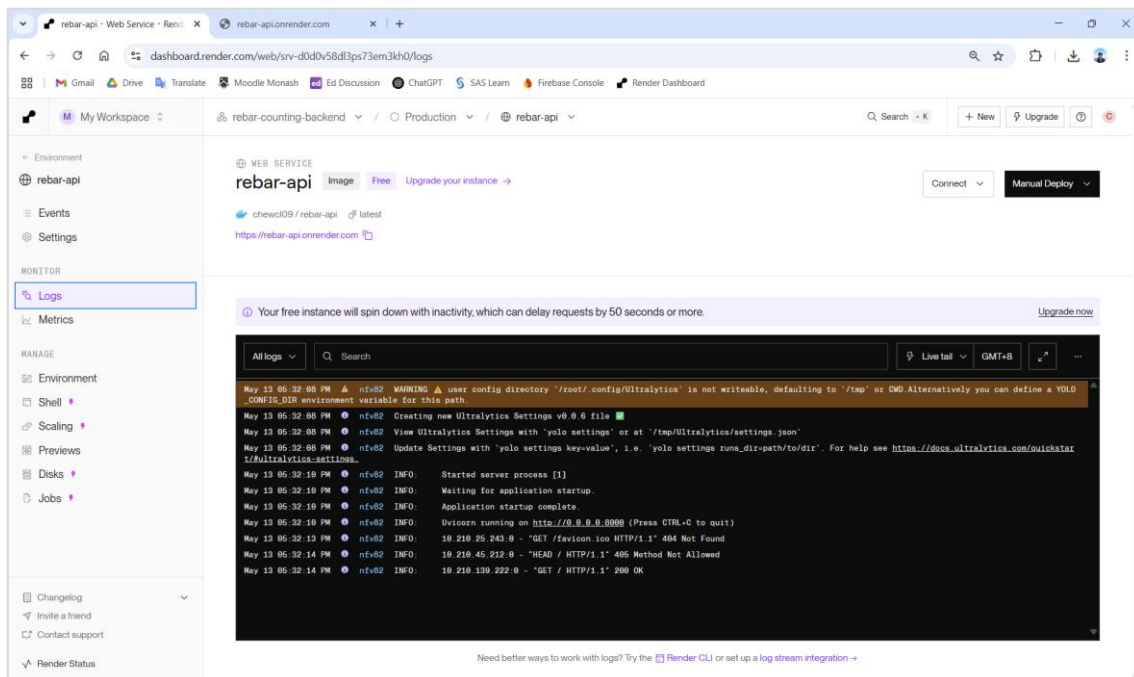


Figure 22 Render.com Deployment Dashboard

Firestore Integration

Firestore is used for structured storage of user profiles, count results, and metadata, while Firebase Storage securely stores original and annotated image files. All data exchanges are encrypted, and access is restricted to authenticated users. ([Firestore Console Link](#))

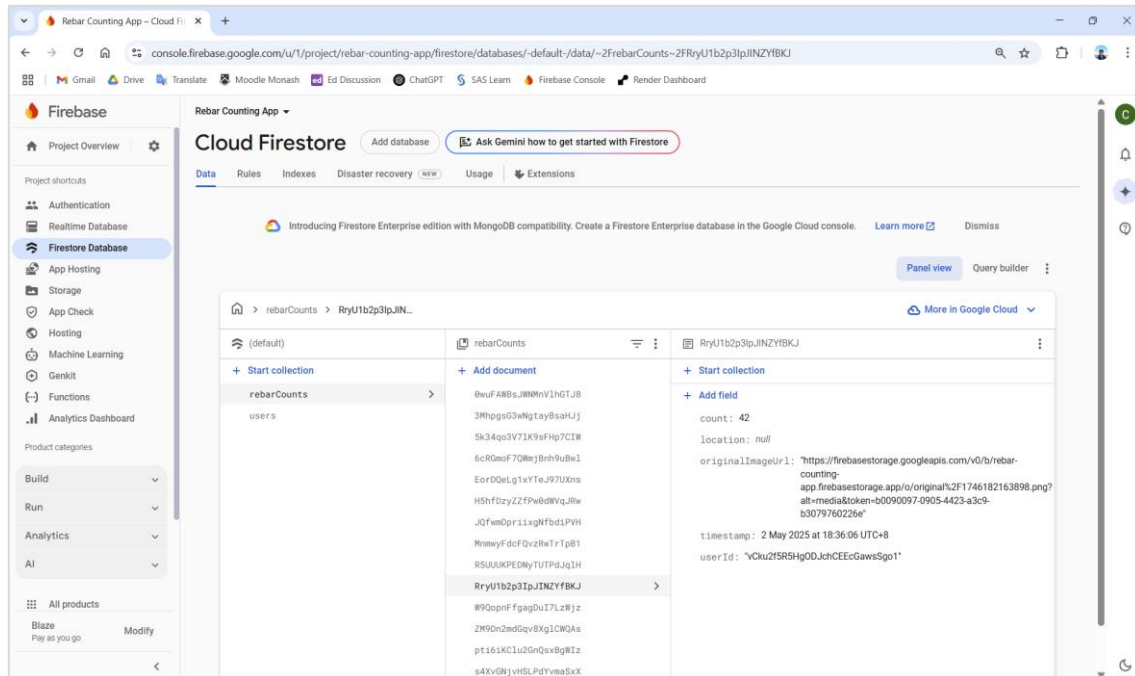


Figure 23 Firebase Console with rebarCounts Collection Sample Document

This backend and cloud stack enables seamless, real-time communication between the mobile app and AI model, supports secure and scalable data management, and ensures operational reliability. For step-by-step backend setup instructions and deployment processes, please refer to the **Technical Guide**.

5.3 Software Quality Assessment

This section critically evaluates the software qualities of the delivered system, discussing both strengths and areas for improvement. Each attribute is supported by evidence from system testing, user feedback, and development documentation. Full test results are available in the **Test Report**.

5.3.1 Robustness and Reliability

The system demonstrates strong robustness across its components, with a focus on fault tolerance and stable operation under adverse conditions. (see **Test Report**, Section 3 White-Box Testing)

- **Input Validation:** All user inputs, including image uploads and annotations, are validated both client-side and server-side to prevent invalid data submission or app crashes.
- **Error Handling:** Comprehensive error handling is implemented at every major integration point. The mobile app displays clear error messages for failed uploads, invalid images, or network interruptions, while the backend returns descriptive HTTP error codes and messages.
- **Recovery and Resilience:** Automated retry mechanisms handle transient network failures, and all failed actions are logged for troubleshooting.

Shortcomings: While the system gracefully handles most common errors, some rare concurrency or cloud service outage scenarios may cause delayed feedback. Further improvements could include implementing offline queuing for uploads or more granular user feedback on cloud errors.

5.3.2 Security and Data Privacy

Security was a primary consideration in both the design and implementation of the system.

- **Secure Data Storage and Transfer:** All communications between the mobile app, backend, and cloud services are encrypted using HTTPS/TLS. User data and images are stored securely in Firebase with strict access control rules, preventing unauthorized access or data leakage.
- **API Security:** The backend validates authentication tokens on every request, and data access is further restricted at the database and storage level.
- **Authentication:** All access to user-specific data and core functionality is restricted via Firebase Authentication, enforcing secure sign-up, login, and session. All web and data security follows guidelines from OWASP and Firebase’s recommended security rules (*Firebase Authentication*, 2025; *Firebase Security Rules*, 2025; *OWASP Top Ten* | *OWASP Foundation*, 2025).

Shortcomings: While core security requirements were met, some potential improvements include introducing two-factor authentication, ensuring all cloud security policies remain up to date.

5.3.3 Usability

Usability was a major focus throughout the development and validation of the system, with design choices aimed at providing a clear and intuitive experience for users in the field.

- **User Interface and Navigation:** The mobile app features a straightforward workflow, guiding users from authentication through image capture, annotation, and result review. Core actions are accessible within a few taps, and consistent UI patterns are used throughout. The *User Guide* provides step-by-step instructions, screenshots, and troubleshooting tips for all features.
- **Accessibility and User Testing:** Usability walkthroughs and user acceptance testing (see *Test Report*, Section 6) confirm that both technical and non-technical users could navigate the app successfully and complete all major tasks without external assistance.

5.3.4 Scalability

The system was designed to support scaling in both user volume and data processing capacity.

- **Cloud-Based Backend:** The FastAPI backend is containerized and hosted on Render.com, supporting automatic scaling to accommodate increased API traffic and concurrent requests.
- **Database and Storage:** Firebase Firestore and Storage are managed cloud services that scale automatically with user growth, dataset expansion, and increased image upload frequency.
- **Model Retraining and Updates:** The modular deployment architecture allows the YOLOv8 model to be updated independently of the mobile app or backend, supporting future improvements or retraining with minimal downtime.
- **Deployment and CI/CD:** Automated deployment pipelines (see *User Guides*, Section 2.6) enable rapid, reliable updates and horizontal scaling as needed.

Shortcomings: Although designed for scalability, actual load testing with hundreds of concurrent users was not performed due to resource constraints. Future work could include stress-testing the full pipeline and monitoring performance under heavy, real-world usage scenarios.

5.3.5 Portability

The system's architecture and technology choices promote portability across a range of hardware and software environments.

- **Mobile App:** Developed in Kotlin using Android Studio and Jetpack Compose, the app is compatible with all modern Android devices (Android 8.0 and above). The APK can be deployed directly or distributed through enterprise channels.
- **Backend and Model:** The FastAPI backend is containerized using Docker, ensuring consistency across local, cloud, and testing environments. Model inference is platform-agnostic if the deployment environment supports Python and PyTorch.
- **Cloud Integration:** Firebase services are accessible globally, allowing seamless operation in any region with internet access. The use of Docker and cloud-based infrastructure enables rapid redeployment and migration to alternative hosting providers if needed.

Shortcomings: The system is currently limited to Android devices, with no IOS support or progressive web app alternative. Backend and model portability are high, but performance on low-resource cloud instances or legacy devices has not been exhaustively tested.

5.3.6 Maintainability

Maintainability has been a guiding principle in both code organization and deployment processes.

- **Modular Design:** The project is structured into clear, independent modules (mobile app, backend API, AI model, cloud integration), facilitating isolated updates and debugging.
- **Documentation:** Comprehensive guides including a Technical Guide, User Guide, and inline comments provide detailed instructions for deployment and usage.
- **Version Control and CI/CD:** All code is maintained in a GitHub repository with branch management and automated CI/CD pipelines for testing and deployment, enabling consistent, traceable updates.

Table 13 Software Quality Attributes – Summary Assessment

Quality Attribute	Assessment	Strengths	Shortcomings / Areas for Improvement
Robustness	Strong	Input validation, error handling, resilience	Rare edge cases, limited offline queuing
Security	Strong	Firebase Auth, HTTPS, secure storage, access control	Lacks 2FA, no formal penetration test
Usability	Strong/Adequate	Intuitive UI, feedback, onboarding, user guide	More in-app help, improved annotation usability
Scalability	Adequate/Strong	Cloud hosting, auto-scaling backend, modular updates	No full-scale load test, stress-testing needed
Portability	Adequate	Android app, Dockerized backend, cloud-based services	Android-only, limited testing on legacy/low-resource devices
Maintainability	Strong	Modular design, documentation, CI/CD, code quality	Some backend/annotation code could be refactored; onboarding time

5.4 Sample Source Code

To demonstrate code quality, inline documentation, and satisfaction of key requirements, representative samples of our source code are provided in the Appendix D – Sample Source Code, which includes:

- **A core backend function** from the FastAPI service that handles image upload, invokes the YOLOv8 model for inference, and returns results to the client, demonstrating requirement satisfaction for automated rebar counting, error handling, and API integration.
- **A client-side function** from the mobile app responsible for invoking the backend /count endpoint and processing the returned result, showing integration with the backend.
- **Supporting validation code** illustrating robust input checking, error messaging, or secure data handling (e.g., image file validation).

These code samples are selected to be representative of the overall project’s coding standards, documentation. For the complete codebase, detailed documentation, and reproducibility instructions, refer to the *Technical Guide* and the project’s GitHub repositories: [Mobile Application Repository](#), [Backend Repository](#), [Model Repository](#).

6. Software and Project Critique

6.1 Evaluation of Project Execution

The execution of this project was characterized by a combination of strong technical delivery, adaptive teamwork, and iterative improvement in response to both challenges and feedback. From initial planning through final deployment, the team maintained a consistent focus on delivering an end-to-end, production-ready rebar counting system, and adapted its approach as new technical and organizational realities emerged.

Strengths

- The project benefited from a clear division of roles and regular, structured communication, which enabled parallel progress on mobile, backend, and model components.
- Technical milestones, including dataset curation, model training, mobile UI development, backend API deployment, and full system integration, were completed on or ahead of schedule for most of the project.
- Flexibility and openness to change were key to success; the team was able to rapidly pivot technology stacks (e.g., moving from Flutter to Kotlin) and adopt new features such as client-side annotation based on emerging requirements and supervisor input.

Challenges

- Synchronizing development across interdependent modules (e.g., aligning mobile app features with backend API schemas and model outputs) was a recurring challenge and required frequent integration testing and cross-functional collaboration.
- The team experienced some workload imbalances during critical phases, such as deployment and CI/CD setup, which were addressed by redistributing tasks and increasing cross-support between members.
- Managing project risks such as delays in dataset annotation and technical uncertainties with cloud deployment demanded regular review and contingency planning.

Adaptation and Improvement

- The team demonstrated resilience in resolving issues as they arose, using regular retrospectives to identify bottlenecks process and refine development priorities.
- Feedback from the supervisor and interim demonstrations was promptly incorporated, leading to tangible improvements in usability, performance, and workflow robustness.

6.2 Outcome vs. Proposal and Deviations

6.2.1 Alignment with Original Proposal

The final project outcomes closely aligned with the objectives and deliverables outlined in the original proposal. Core goals were achieved as planned, including an AI-driven rebar counting system, mobile application for image capture and annotation, cloud-based inference, and comprehensive data management. Major milestones, such as dataset preparation, model training, backend deployment, and mobile app development, were completed in accordance with the project schedule, and all critical functional and non-functional requirements were satisfied.

Features and deliverables that remained consistent with the proposal include:

- End-to-end workflow from image capture and annotation to automated rebar counting and historical result tracking.
- Integration of Firebase Authentication, Firestore, and Storage for secure, scalable data management.
- Deployment of a YOLOv8-based model via a FastAPI backend, with performance optimized for real-time feedback.

6.2.2 Key Deviations and Their Rationale

While the project maintained strong alignment with its initial scope, several important deviations were implemented to address technical challenges and user needs. Full details of software methodology and implementation changes are documented in *4.5 Deviation from Initial Design*.

Table 14 Summary of Major Deviations from Original Proposal

Deviation	Original Plan	Actual Implementation	Rationale/Impact
Model Framework	Faster-RCNN	YOLOv8	Higher inference speed, competitive precision, great scaling potential.
Mobile App Framework	Flutter for cross-platform development	Kotlin/Jetpack Compose (Native Android)	Improved Firebase integration, annotation capability, and UI performance.
Model Inference Location	On-device or hybrid	Fully cloud-based via FastAPI backend	Ensured consistency, easier updates, and scalable inference regardless of device hardware.
Image Annotation Tool	Not included	Client-side annotation feature added	Enhanced usability and counting accuracy in complex images; highly valued by users.
DevOps/Deployment Pipeline	Basic cloud deployment	Advanced CI/CD with GitHub Actions, Docker	Automated builds and deployments, reduced downtime, improved release turnaround.
System Resilience & Error Handling	Not explicitly emphasized	Robust validation, error messages, fallback	Increased reliability, particularly under low-connectivity or input error scenarios.
Project Methodology	Agile only	Hybrid Agile-Waterfall	Better fit for academic schedule and supervisor-driven feedback cycles (see <i>Project Management Report</i>).

These changes and adaptations were implemented in response to real-world constraints, technical learnings, and ongoing feedback from users and the project supervisor. The result was a system that fulfilled the project's core objectives while offering greater usability, resilience, and deployment flexibility than initially anticipated. Overall, the project's outcomes are well-aligned with the initial proposal, with deviations carefully justified and documented to enhance both technical feasibility and end-user value.

6.3 Project Management Reflection

Effective risk and schedule management, clear role allocation, and responsive stakeholder engagement were central to the project's successful delivery (Shastri et al., 2021). The team's ability to adapt its management approach, coupled with regular communication and critical self-assessment, resulted in a positive project experience and a well-executed final product. For a detailed breakdown of management practices, sprint outcomes, and risk strategies, refer to the *Project Management Report*.

Risk Management

A comprehensive risk register was developed at the outset and continuously updated as the project evolved. Risks related to data collection, model accuracy, cloud deployment, and mobile device compatibility were identified and tracked, with comprehensive mitigation strategies. The risk register (see Appendix E – Risk Register) details all identified risks, likelihoods, impacts, mitigation actions, and status. This structured approach helped avoid major delays and ensure timely delivery of critical deliverables.

Schedule and Scope Management

The project timeline was initially mapped out using a detailed Gantt chart (see Appendix F – Gantt Chart, and execution was organized around iterative sprint cycles. Progress was monitored using milestone checklists and weekly reviews, with progress tracking (see Appendix G – Sprint Cycle Overview). Scope was actively managed by prioritizing core functionality and deferring non-essential features, ensuring that project objectives remained realistic and achievable within resource and time constraints.

Role Allocation and Teamwork

Team members were assigned clear primary responsibilities (e.g., mobile development, model training, backend/API, testing), but collaboration and cross-support were emphasized throughout the project. When bottlenecks or skill gaps were identified, tasks were redistributed, and knowledge sharing was facilitated. This flexible approach enabled the team to address critical-path challenges efficiently and maintain steady progress even during peak workload periods.

Stakeholder Engagement

Regular engagement with the project supervisor provided essential feedback and validation, influencing several key decisions such as the adoption of the annotation tool and the choice of cloud inference. Interim demonstrations and walkthroughs allowed for early identification of usability and technical issues, ensuring that the system remained aligned with user needs and expectations. Stakeholder feedback was incorporated iteratively, contributing to the final product's robustness and usability.

Methodology Impact

The team initially adopted an Agile methodology but shifted to a hybrid approach that combined sprint-based development with milestone-driven reviews. This transition allowed the team to better accommodate academic deadlines and stakeholder input while preserving the benefits of iterative prototyping and feedback.

6.4 Lessons Learned and Recommendations

Reflecting on the project journey, several key lessons and actionable recommendations have emerged that would benefit future teams undertaking similar AI-driven software development projects:

Lessons Learned:

- Early and regular integration of system components is essential for detecting compatibility issues, technical misalignments, and workflow gaps. Delaying integration increases the risk of discovering critical problems late in the development cycle.
- Separating experimental model research from core application development allows both streams to progress without blocking each other, reducing deployment conflicts and rework.
- Clearly defining and enforcing a minimum viable product (MVP) boundary helps keep the project focused, prevents feature creep, and reduces technical debt accumulation.

Recommendations:

- Prioritize early integration milestones across all technical components to surface and address cross-module issues as soon as possible.
- Maintain clear separation between experimental model development and core application implementation to avoid delays and reduce deployment risk.
- Strictly enforce MVP boundaries, ensuring that all additional features or refinements are evaluated for necessity and potential project impact before inclusion.

By applying these lessons, future teams can enhance project efficiency, reduce risk, and increase the likelihood of delivering robust, high-quality solutions within constrained timelines.

7. Conclusion

7.1 Summary of Key Findings

This report presented the development and evaluation of an AI-powered mobile application for automated rebar counting, aimed at improving accuracy, efficiency, and traceability in the construction supply chain. The system integrates a native Android mobile interface, a FastAPI backend, and a YOLOv8 deep learning model deployed on the cloud. Key methodologies, from dataset preparation to system deployment, were described in detail, with validation results confirming strong performance under real-world conditions.

The implemented system successfully addressed the original goals of enabling users to capture rebar images, perform accurate counting via AI inference, and store results securely through Firebase integration. The Technical Guide and Test Report demonstrate that the development process is reproducible, and the system performs consistently across diverse scenarios.

7.2 Overall Evaluation of Project Outcomes

The project met all functional and non-functional requirements outlined in the initial proposal, with enhancements made based on stakeholder feedback. Key achievements include:

- **High model accuracy**, with precision of 0.983 and counting accuracy of 96.82%.
- **Low-latency performance**, averaging under 2.5 seconds per inference.
- **Seamless integration**, with cloud-based data storage and user authentication.
- **Robust usability**, supported by clear workflows, annotation tools, and a traceable results history in the **mobile application deliverable**.

While exact-match counting remains an area for future enhancement, the system demonstrated reliable, practical performance suitable for field deployment. All deviations from the initial design were justified by performance trade-offs or usability gains.

7.3 Final Thoughts and Recommendations

The success of this project illustrates the potential of integrating mobile computing and AI for practical applications in industrial contexts. By addressing real-world challenges such as occlusion, user accessibility, and performance scalability, the project delivers a working prototype that can be confidently extended for commercial use.

Future work should focus on expanding the dataset, improving exact-match detection, supporting offline inference, and conducting broader stakeholder trials. With these refinements, the system can become a dependable tool for construction teams and inventory managers seeking to automate manual tasks with AI-powered precision.

8. Appendices

Appendix A – System Latency Performance

Table 15 End-to-End System Latency Performance (Measured on 100 Test Submissions)

Metric	Value
Number of Images Tested	100
Minimum Latency	1.8 seconds
Average Latency	2.4 seconds
Median Latency	2.2 seconds
90th Percentile Latency	3.9 seconds
Maximum Latency	4.6 seconds
Percentage Within 5 Seconds	100%

Notes:

- Latency measured from image submission on the mobile app to result display.
- All test runs conducted under typical network conditions (Wi-Fi, average 30–50 Mbps).
- No instances of latency exceeding the 5-second requirement were observed.

Appendix B – Model Performance

Table 16 Object Detection Model Performance Metrics (on Validation Set)

Metric	Value
Precision	0.983
Recall	0.971
F1-score	0.977
mAP@0.50	0.988
mAP@0.50–0.95	0.743
Validation Images Processed	135
Mean Absolute Error (MAE)	4.393 rebars/image
Counting Accuracy (COCO-style)	96.82%
Exact-Match Accuracy	26.67%

Appendix C – Requirement Traceability Matrix

Table 17 Requirement Traceability Matrix

Req. ID	Description	Category	Type	Status
R1	Allow users to input rebar inventory image.	Application	FR	Completed
R2	Automated rebar counting using deep learning model.	Deep Learning	FR	Completed
R3	System should respond with rebar count output within 5 seconds.	Optimisation	NFR	Completed
R4	Ensure secure access with role-based permissions.	Security	NFR	Completed
R5	The system shall have a user-friendly interface that minimizes steps required for inventory-related tasks.	User Interface	NFR	Completed
R6	The model should be robust under different lighting conditions, angles, and environments.	Optimisation	NFR	Completed
R7	The system should be able to process and predict rebar count using mobile images.	Optimisation	NFR	Completed
R8	Users should be able to annotate images to enhance counting accuracy.	Annotation	NFR	Completed
R9	Users should be able to view the history of previous rebar counts.	Data Management	FR	Completed
R10	All user data and image submissions must be stored securely and backed up in the cloud.	Data Security	NFR	Completed
R11	The system shall provide clear error messages and support recovery from common user mistakes and failures.	User Experience	NFR	Completed
R12	Support for handling varied image resolutions and sizes from multiple Android devices.	Compatibility	NFR	Completed
R13	The backend must support automated deployment (CI/CD) and scaling for multiple concurrent requests.	Infrastructure	NFR	Completed

Appendix D – Sample Source Code

Core Backend Function and Model Loading Code

```
# POST endpoint that receives an uploaded image file and returns a rebar count + annotated image
@app.post("/count/")
async def count(image: UploadFile = File(...)):
    # Log the incoming request details
    logger.info(f"Received request to /count/ with file: {image.filename},
                content-type: {image.content_type}")

    # Validate supported image types: only PNG and JPEG are allowed
    if image.content_type not in ["image/png", "image/jpeg", "image/jpg"]:
        logger.error(f"Invalid content type: {image.content_type}")
        raise HTTPException(status_code=400, detail="Only PNG or JPEG images are allowed")
    try:
        # Read the image bytes from the uploaded file
        img_bytes = await image.read()

        # Log and decode the image into a PIL RGB format
        logger.info("Decoding image")
        pilImg = Image.open(io.BytesIO(img_bytes)).convert("RGB")
        logger.info("Image decoded successfully")

        # Run YOLO model prediction on the image
        logger.info("Running YOLO prediction")
        prediction = model.predict(pilImg)          # This would be mocked during unit tests
        rebar_count = len(prediction)              # Count detected rebar instances
        logger.info(f"Prediction complete, rebar count: {rebar_count}")

        # Annotate the image with the prediction results
        ok, buf = draw_image(pilImg, prediction)

        # If image annotation failed, raise server error
        if not ok:
            raise HTTPException(500, "Failed to encode image")

        # Convert the annotated image to base64 for transmission
        b64 = base64.b64encode(buf.tobytes()).decode()

        # Return the rebar count and base64-encoded annotated image
        return {
            "rebarCount": rebar_count,
            "annotated_image": f"data:image/jpeg;base64, {b64}"
        }
    except Exception as e:
        # Catch any exception and return a client-friendly error message
        logger.error(f"Error processing image: {str(e)}")
        raise HTTPException(status_code=400, detail=f"Invalid image file: {str(e)}")
```

Test Validation Function

```
def test_tc_wb_04_valid_jpeg(self):
    """TC-WB-04: valid JPEG upload returns 200 + JSON with rebarCount and annotated_image"""

    # Open the sample JPEG image in binary read mode
    with open(self.sample_image, "rb") as f:
        data = f.read()

    # Prepare the multipart/form-data payload with the image
    files = {"image": ("rebar.jpg", data, "image/jpeg")}

    # Send a POST request to the /count/ endpoint with the image file
    resp = self.client.post("/count/", files=files)

    # Assert that the response status code is 200 OK
    self.assertEqual(resp.status_code, 200, resp.text)

    # Parse the JSON response body
    j = resp.json()

    # Check that the response contains the expected keys
    self.assertIn("rebarCount", j)      # The rebar count field must exist
    self.assertIn("annotated_image", j)  # The base64-encoded image must exist

    # Ensure the mocked model returned the expected count of 0
    self.assertEqual(j["rebarCount"], 0)

    # Check that the annotated image is a base64-encoded JPEG string
    self.assertTrue(j["annotated_image"].startswith("data:image/jpeg;base64,"))

    # Print results for manual verification/debugging
    print(f"TC-WB-04 rebarCount: {j['rebarCount']}")
    snippet = j["annotated_image"][0:80] + "..."
    print(f"TC-WB-04 annotated_image (base64 snippet): {snippet}")

if __name__ == "__main__":
    unittest.main()  # Automatically discovers and runs test_* functions
```

Mobile Application Function (API Calling)

```
// Singleton object that provides a configured Retrofit instance for making API calls
object RetrofitProvider {
    // OkHttpClient with logging and a generous read timeout (5 minutes)
    private val client = OkHttpClient.Builder()
        .addInterceptor(logging) // Attach the logging interceptor
        .readTimeout(5, TimeUnit.MINUTES) // Time to wait for server to send full response
        .build()

    // Lazy-initialized Retrofit API interface instance
    val api: RebarApi by lazy {
        Retrofit.Builder()
            // Base URL of the backend API
            .baseUrl("https://rebar-count.onrender.com")
            // Use Moshi for JSON serialization
            .addConverterFactory(MoshiConverterFactory.create())
            .client(client) // Attach the configured OkHttpClient
            .build()
            // Create implementation of RebarApi
            .create(RebarApi::class.java)
    }
}

// Example function call
val apiResp = RetrofitProvider.api.countRebars(part)
```

Appendix E – Risk Register

Table 18 Risk Register

No.	Rank	Risk	Description	Category	Triggers	Root Cause	Potential Responses	Risk Owner	Probability	Impact	Status	Score
R01	1	Poor image quality affecting model accuracy	Low-quality images could result in inaccurate rebar counting, affecting the system's performance	Technical	Low-quality or distorted images captured by the system	Poor lighting, camera misalignment, low resolution, environmental interference (e.g., dust, debris)	Replace or upgrade imaging equipment (e.g., higher resolution cameras).	Technical Developer	High	Very High	Triggered	63
R02	4	Algorithm fails to handle varying environmental conditions	The algorithm may struggle to generalize under different environmental conditions	Technical	Model performs inconsistently in different conditions	Lack of diverse environmental training data	Expand dataset with diverse scenarios, perform rigorous testing	Technical Lead	Medium	Very High	Monitored	45
R03	7	Client dissatisfaction with system performance	The client may not be satisfied with the system's accuracy, speed, or usability	Stakeholder	Negative feedback during testing or after deployment	Performance fails to meet client's accuracy or usability expectations	Engage in regular client communication, perform user acceptance testing (UAT)	Project Manager	High	Medium	Monitored	38
R04	5	Underperformance of AI model	The AI model may consistently fail to meet accuracy benchmarks	Technical	Model consistently fails accuracy benchmarks	Algorithm complexity or insufficient testing	Fine-tune the model, explore alternative approaches	Quality Assurance	Medium	High	Triggered	42
R05	11	Team member absence	A team member's unexpected absence could cause delays in project deliverables	Project Management	Team member unable to continue contributing	Personal emergencies, illness, or external factors	Cross-train team members, assign backup responsibilities	Project Manager	Low	Medium	Monitored	15
R06	9	Delay in project milestones	Key milestones may be missed, delaying the overall project timeline.	Project Management	Missed deadlines for key tasks or deliverables	Underestimating task durations, dependencies	Use Gantt chart for detailed planning, adjust timelines as needed	Project Manager	Low	High	Triggered	33

R07	2	Insufficient training data for model	The model may not generalize well due to insufficient or biased training data	Technical	Model performance below threshold during testing	Limited dataset, biases dataset, or insufficient variation in training scenarios.	Expand data collection, apply data augmentation techniques	Technical Lead	High	Very High	Monitored	60
R08	12	Client's workers not using Android phones	Workers using non-Android phones could limit the effectiveness of the mobile app	Stakeholder	Client workers prefer non-Android devices	Lack of standardized device policy among workers	Explore iOS development or supply Android devices	Project Manager	Very Low	Low	Monitored	3
R09	8	Security vulnerability in system	A security breach could compromise data integrity and system reliability	Technical	Breach during testing or post-deployment	Insufficient security measures in code or data handling	Implement encryption, conduct security audit	Technical Developer	Medium	High	Monitored	35
R10	10	Communication breakdown within the team	Miscommunication or lack of clarity in task allocation could delay deliverables	Project Management	Missed meetings, lack of clarity in task allocation	Poor internal communication, lack of clear reporting	Regular check-ins, use project management tools	Project Manager	Low	Medium	Monitored	20
R11	6	Inconsistent testing and quality checks	Quality assurance procedures not followed consistently, leading to undetected system flaws	Quality Assurance	Irregular or skipped testing during the project lifecycle	Incomplete or inconsistent testing procedures	Establish clear and consistent testing schedules	Quality Assurance	Medium	High	Monitored	40
R12	3	Project scope creep	The project scope may expand beyond its initial boundaries, leading to delays and resource strain	Project Management	Adding unplanned features or modifications during development	Lack of clear scope management and boundary control	Implement clear scope management processes, evaluate all change requests	Project Manager	Medium	High	Monitored	45

Appendix F – Gantt Chart

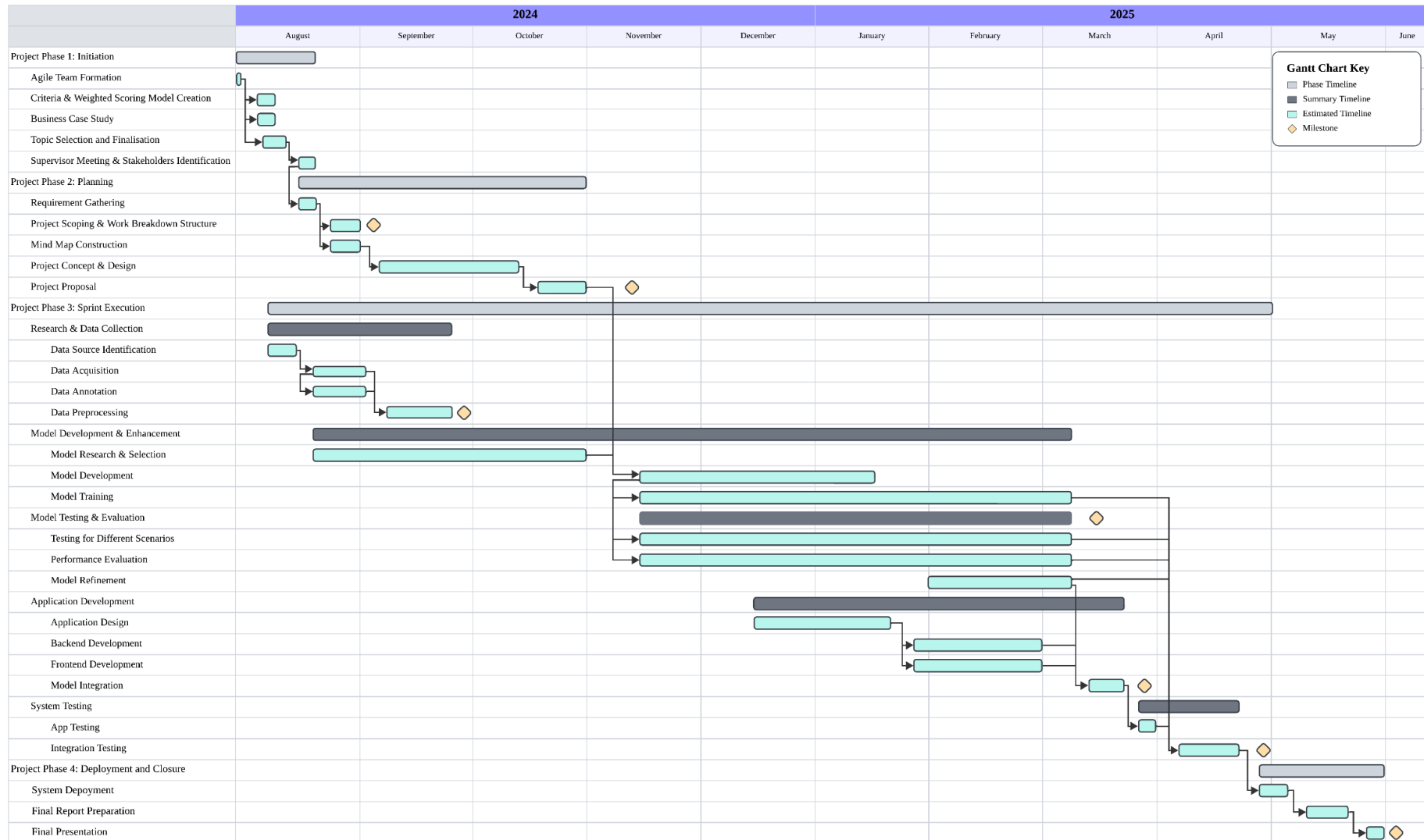


Figure 24 Gantt Chart

Appendix G – Sprint Cycle Overview

Table 19 Sprint Cycle Overview

Sprint	Key Goals	Main Tasks	Expected Outcomes	Next Steps	Status
Sprint 1	1. Finalize project scope and initial requirements 2. Set up baseline app code	- Collect initial rebar image dataset - Conduct literature review on rebar detection algorithms - Set up Android project - Create user login screen (UI)	- Project scope clearly defined - Rough dataset of 100 rebar images gathered - Android project skeleton created - User login screen functional, though not fully tested	- Prepare a refined dataset for the next sprint's AI model exploration - Implement basic QA tests for login flow	Completed
Sprint 2	1. Develop baseline AI model 2. Integrate model with mobile app	- Label rebar images (annotation) - Train baseline faster-RCNN model on 70% training set - Implement model inference endpoint (backend) - Mobile app captures image	- Annotate dataset with bounding boxes - Baseline faster-RCNN model achieve ~70% detection accuracy - Basic app-backend integration complete - Users can capture images through the app	- Improve data augmentation for next model iteration - Address poor detection of overlapping rebars - Plan to add real-time feedback for user image capture	Completed
Sprint 3	1. Refine AI model for higher accuracy 2. Implement real-time feedback	- Experiment with hyperparameter tuning (batch size, learning rate) - Incorporate annotation mode for complex rebar stacks - Begin implementing rebar count overlay on captured images	- AI model accuracy improves to ~85% - Include annotation mode, letting users highlight problem areas - Basic overlay displays rebar count on the mobile screen	- Conduct mid-sprint user test to gauge annotation usability - Expand dataset with more diverse rebar conditions (rust, varied lighting) - Prepare for mid-semester review	Completed
Sprint 4	1. Final feature polishing and bug fixes 2. Prepare for deployment	- Refactor UI for consistency - Final QA testing and bug fixing - Optimize model inference performance	- Smooth user experience across the app - App and model ready for final evaluation	- Write closure documentation - Finalize presentation slides	Completed

9. References

- Armbrust, M., Fox, A., Griffith, R., Joseph, A. D., Katz, R., Konwinski, A., Lee, G., Patterson, D., Rabkin, A., Stoica, I., & Zaharia, M. (2010). A view of cloud computing. *Commun. ACM*, 53(4), 50–58. <https://doi.org/10.1145/1721654.1721672>
- Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). *End-to-End Object Detection with Transformers* (No. arXiv:2005.12872; Issue arXiv:2005.12872). arXiv. <http://arxiv.org/abs/2005.12872>
- Firebase Authentication*. (2025). Firebase. <https://firebase.google.com/docs/auth>
- Firebase Security Rules*. (2025). Firebase. <https://firebase.google.com/docs/rules>
- info@dbmsteel.com. (2023, January 11). *Rebar Counter System For Rebar Production Line*. DBM Metallurgical Intelligence. <https://dbmsteel.com/rebar-counting-system-for-rebar-production-line/>
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux J.*, 2014(239), 2:2.
- OpenCV: Geometric Image Transformations*. (n.d.). Retrieved June 5, 2025, from https://docs.opencv.org/4.x/da/d54/group__imgproc__transform.html
- OpenCV: Template Matching*. (2024, October 24). https://docs.opencv.org/4.x/d4/dc6/tutorial_py_template_matching.html
- OWASP Top Ten | OWASP Foundation*. (2025). <https://owasp.org/www-project-top-ten/>
- Reason, J. (1990, October 26). *Human Error*. Higher Education from Cambridge University Press; Cambridge University Press. <https://doi.org/10.1017/CBO9781139062367>
- Reis, D., Kupec, J., Hong, J., & Daoudi, A. (2024). *Real-Time Flying Object Detection with YOLOv8* (No. arXiv:2305.09972). arXiv. <https://doi.org/10.48550/arXiv.2305.09972>
- Ren, S., He, K., Girshick, R., & Sun, J. (2016). *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks* (No. arXiv:1506.01497; Issue arXiv:1506.01497). arXiv. <http://arxiv.org/abs/1506.01497>

Shastri, Y., Hoda, R., & Amor, R. (2021). The role of the project manager in agile software development projects. *Journal of Systems and Software*, 173, 110871. <https://doi.org/10.1016/j.jss.2020.110871>

What is data augmentation? | IBM. (2024, May 7). <https://www.ibm.com/think/topics/data-augmentation>

Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2021). *Deformable DETR: Deformable Transformers for End-to-End Object Detection* (No. arXiv:2010.04159; Issue arXiv:2010.04159). arXiv. <http://arxiv.org/abs/2010.04159>